

CargoExpress PH — Chapter 2 Database and Process Handoff

Purpose of this file. This is a self-contained handoff for revising Chapter 2 of the CargoExpress PH manuscript. Another reviewer or ChatGPT session should be able to revise the chapter from this file alone. It compares the manuscript with the **live Supabase database**, the **repository source code**, and **isolated tests**. Nothing in the system, database, migrations, or manuscript was changed to produce it.

Inspection date: 24 September 2026 (Asia/Manila). **Manuscript inspected:** `Chapter-2-CHECKED-1-1.pdf`, 107 PDF pages. The file includes Chapter 2, Chapter 3, the Action Plan, References, and Appendices A–D. **Repository:** branch `main`, commit `cd42145680d4a94c5b2442a24c0f57795a67e153` ("Preserve applied support chat migrations before rollback", 24 Sep 2026 16:46 +08:00).

How to read the evidence labels in this file

- LIVE** — confirmed by read-only catalog/metadata queries against the linked production Supabase project on the inspection date.
- SOURCE** — read from repository code or migrations, but not executed.
- TEST** — confirmed by an existing isolated test (an in-memory PGlite database or a source-contract script). These tests never touch production.
- UNVERIFIED** — could not be confirmed. Do not present it as fact in the manuscript.

1. Scope, baseline, and access limitations

1.1 What was inspected

Item	Result	Evidence
Repository state	<code>main</code> at <code>cd42145</code> . Uncommitted: <code>CHAPTER_2_UPDATE_REVIEW.md</code> (an older review of a <i>different</i> 111-page PDF, do not use) and this file. No code or migration changes were uncommitted.	<code>git status</code>
Supabase project	Linked project reference <code>duigaivxgxl njmfienhg</code> . No credentials are reproduced in this file.	<code>supabase/.temp/project-ref</code>
Live database access	Succeeded. Read-only queries were made through the Supabase CLI (<code>supabase db query --linked, supabase migration list --linked, supabase functions list</code>). Only catalog metadata and aggregate counts were read. No customer rows, names, emails, or amounts per person were read. Nothing was written.	CLI output
Local vs remote migrations	Identical. All 219 local migration files (<code>supabase/migrations/</code>) are recorded as applied remotely, in the same order, with no missing or extra versions. The last applied migration is <code>20260924083945_restore_original_support_chat.sql</code> .	LIVE
Edge Functions	21 are deployed and ACTIVE. The repository has 20. The extra deployed function, <code>support-bot</code> , belongs to a support-chat change that was reverted. It has no local source and no frontend caller. It is an orphan and should not be documented as a feature. Whether each deployed function's code is byte-identical to the local source was not verified.	LIVE + SOURCE
Scheduled jobs	10 <code>pg_cron</code> jobs are active. All ran successfully in the last 2 days (weekly job last ran 20 Sep). All recent outbound Edge Function calls in <code>net._http_response</code> returned HTTP 200. Both required Vault secret <code>names</code> exist; their values were not read.	LIVE
Live data volume (aggregate only)	9 bookings, 3 trips, 7 profiles, 17 payment transactions. No booking is currently Cancelled or flagged for coverage review.	LIVE
Tests run for this handoff	5 isolated tests, all passed (Section 12.3). No browser or end-to-end test was run.	TEST

1.2 Separation of evidence layers

- Verified live database state:** the table list, columns, types, nullability, defaults, primary/foreign/unique keys, CHECK constraints, unique indexes, triggers, function definitions, RLS policy names, cron jobs, storage buckets, and deployed Edge Function list (Sections 3, 5, 10).
- Current repository implementation:** React pages/routes, `src/lib/database.js`, `src/lib/supportChatEngine.js`, `src/lib/perTripSalesReport.js`, and Edge Function source. The migrations and live database match, so SQL behavior described here is live. Frontend behavior is **SOURCE** only: the deployed Vercel build was not inspected.
- Local changes not yet deployed:** none found for SQL. Frontend deployment was not checked.
- Proposed or draft features:** none are described as implemented in this file.

1.3 Platform-managed schemas (not business tables)

The database also contains Supabase-managed schemas: `auth` (login accounts, sessions), `storage` (photo files; buckets `cargo-photos` private, `company-assets` public), `realtime`, `cron`, `net`, `vault`, `extensions`, and `supabase_migrations`. These are platform infrastructure. They should be described in prose as integrations ("Supabase Authentication", "Supabase Storage"), **not** counted as CargoExpress business tables.

1.4 Scope decision: core business tables

The owner chose to present the **core business tables** in Chapter 2 rather than every technical table. The manuscript's Database Design introduction (PDF p. 51) says: *"The following tables present the database structure of the CargoExpress PH system and describe the information stored and managed by each table."* That sentence implies completeness. No adviser instruction in the PDF explicitly requires every table. **No conflict was found**, but the introduction must be reworted so a selected set is not presented as the complete physical database. Suggested wording is in Section 13, item 1.

2. Existing Chapter 2 structure and exact formatting templates

2.1 Structure of the manuscript (PDF page numbers)

PDF pages	Content
1–5	Chapter 2 — PRESENTATION OF THE PRESENT AND PROPOSED SYSTEM. A. PRESENT SYSTEM: System Narrative, Constraints
6	Figure 11. Context Diagram of Present System
7–8	Event List (9 events), Figures 12–20
9–13	B. PROPOSED SYSTEM: System Rules
14	Figure 21. Context Diagram of Proposed System
15–18	Event List (16 events), Figures 22–37
19–47	Process Specification , items 1–27
48–50	Figures 38–40, Program Hierarchy (Public, Customer, Admin)
51–62	Database Design , Tables 1–14
63–72	Test Data/Cases 1–10
73	Revision of the System; Technical Requirements
74–77	Hardware, Software, Peopleware
77–79	Cost-Benefit Analysis, Table 15, Benefits
80–93	Interface Design, Previews 11–25
94–96	Chapter 3
97–99	Action Plan
100–101	References
102–107	Appendices A–D (Appendix D, User's Guide, is empty on p. 107)

The existing Database Design order is: 1 Activity_logs, 2 Announcements, 3 Chat_messages, 4 Company_information, 5 Contact_inquiries, 6 Conversations, 7 Customer_feedback, 8 Notifications, 9 Order_status_events, 10 Orders, 11 Payment_transactions, 12 Profiles, 13 Trips, 14 Payment_refunds (alphabetical, except Payment_refunds added last). There is **no separate "Data Dictionary" heading**. The Database Design tables serve as the data dictionary. There is **no ERD** in the PDF. Figures 22–37 are event-level data-flow drawings.

All pages were readable. Line/arrow drawings do not extract as text, so diagrams were checked from the rendered pages. The typographical errors noted on those pages are listed in Section 11.

2.2 Template A — Database table description (PDF p. 51)

The manuscript writes one caption line directly above each grid, with no separate "Purpose" label:

Table 1. Activity_logs: This table tracks the actions performed by admins for system auditing.

Pattern: **Table [n]. [Table_name]: This table [purpose sentence]**. The table name is written with an initial capital and underscores (e.g., **Order_status_events**). The name is bold in the PDF; the rest of the line is not.

2.3 Template B — Data dictionary grid (PDF p. 51)

Three columns only. Use the page-51 header capitalization consistently. Later pages switch to **Field name / Data type**; standardize to page 51.

Field Name	Data Type	Description
id	uuid	Unique identifier for each activity log entry.
admin_id	uuid	Stores the identifier of the admin who performed the action.

Style rules observed: field names in plain lowercase snake_case; PostgreSQL short type names (**uuid, text, varchar, int4, numeric, bool, jsonb, timestampz, date**); descriptions are one sentence starting with a verb ("Stores...", "Records...", "Indicates...", "Unique identifier for..."), ending with a period. The grid has **no** key, nullability, or default columns. This handoff puts those facts in a short paragraph *after* each grid. The manuscript can do the same, or omit them.

2.4 Template C — Process specification (PDF pp. 19–47)

Numbered title, **Begin/End**, indented actor or system steps, nested **If / Else If / Else / End If**. Button labels and on-screen messages appear in straight double quotes. The manuscript uses **no** Actor/Input/Output headings inside a process. Faithful example (PDF p. 19):

```
1. Home and About Us
Begin
  Display Company Overview
  If Visitor selects "Sign In"
    Redirect to Login Page
  End If
  If Visitor scrolls to Contact Us
    Display Contact Form
    Visitor enters Name, Mobile Number, Email Address, and Message
    Visitor optionally checks "Receive announcement updates by email"
    Click "Send Message"
    If input is valid
      System records the inquiry
      Display "Message sent! We will contact you soon."
    Else
      Display field validation errors
    End If
  End If
End
```

Section 8 keeps this exact style. The detailed backend trace (actor, trigger, inputs, preconditions, authorization, reads/writes, failures, background work) is in Section 7, outside the pseudocode.

3. Complete application-table inventory (LIVE)

The live database has **29 application tables**: 27 in **public** and 2 in **private**. There are **no views or materialized views** in **public** or **private** (LIVE). Retention rules are detailed in Section 10.

Placement codes: **A** = core business table (main chapter); **B** = supporting (brief paragraph or appendix); **C** = technical/infrastructure (architecture summary); **D** = uncertain.

#	Table (schema)	Plain-language purpose	Business module	Written by	Read by
1	profiles (public)	One account's name, contact, address, and role	Accounts	Signup trigger <code>handle_new_user</code> ; customer profile page; <code>guard_profile_write</code> blocks role changes	Every page; RI workers
2	trips	One scheduled van trip on a route, with planned and actual times	Trips	Admin Create Trip; <code>reschedule_trip</code> ; trip status updates guarded by triggers	Schedules, bo public tracking
3	orders	One booking: contacts, cargo, status, and cached payment summary	Bookings	Customer booking; admin walk-in; pickup/delivery RPCs; trip cascade trigger	Nearly every p workers
4	order_status_events	One status change in a booking's timeline	Tracking	Trigger <code>log_order_status_event</code> only	Public/custom expiry rule
5	payment_transactions	One confirmed payment (full or partial)	Payments	<code>record_pickup_payment</code> , <code>record_delivery_payment</code> , <code>record_additional_payment</code> , PayMongo webhook reconciliation	Payment histo
6	payment_refunds	One return of money, via PayMongo or recorded manually	Refunds	<code>paymongo-refund</code> , <code>record-manual-refund</code> Edge Functions; webhook	Order money l reports, recov
7	cancellation_settlements	Current admin decision about money held for a cancelled booking	Cancellation	<code>record_cancellation_settlement_decision</code> , <code>amend_cancellation_settlement_decision</code>	Settlement su
8	cancellation_settlement_history	Immutable log of each settlement decision/amendment	Cancellation audit	Same settlement RPCs	<code>get_cancell</code>

#	Table (schema)	Plain-language purpose	Business module	Written by	Read by
9	<code>payment_attempts</code>	One GCash checkout attempt (not a payment)	Payments	<code>paymongo-create-payment</code> ; webhook/return reconciliation	Payment retur
10	<code>announcements</code>	One admin notice (public or email-only) with optional link	Announcements	Admin Announcements page; trip notices	Customer hon
11	<code>announcement_email_broadcasts</code>	Delivery job summary for one announcement email	Email delivery	Publish flow; <code>broadcast-announcement</code> worker	Admin delivery
12	<code>announcement_email_recipients</code>	Per-recipient email delivery/retry state	Email delivery	Broadcast worker	Worker, admin
13	<code>email_subscriptions</code>	Whether an email address wants announcement emails	Preferences	Registration, contact form, admin, <code>unsubscribe-announcements</code>	Broadcast reci
14	<code>notifications</code>	One in-app alert to one user	Notifications	Many SQL triggers (<code>notify_*</code>)	Notification pe
15	<code>notification_delivery_jobs</code>	Push-notification delivery queue	Push delivery	Enqueue trigger; <code>process-push-deliveries</code>	Push worker, f
16	<code>user_device_tokens</code>	Registered browser/device push tokens	Push delivery	Browser registration RPCs	Push worker
17	<code>conversations</code>	The customer's support thread and its state	Support	Chat triggers; admin/customer status actions; stale-resolve job	Support chat,
18	<code>chat_messages</code>	One message in a support thread	Support	Customer, admin, and menu-bot inserts (bot messages written by the customer's session)	Chat pages, ui
19	<code>contact_inquiries</code>	One public contact-form submission and handling state	Public inquiries	<code>submit-inquiry</code> Edge Function; admin Inquiries page	Admin Inquirie
20	<code>customer_feedback</code>	One rating/review for a delivered booking	Feedback	Customer order page	Admin modera (<code>get_public</code>
21	<code>company_information</code>	Single settings record: company details, coverage, default rate and capacity	Company settings	Admin Company Information page	About page, C pricing fallback <code>global_pric</code>
22	<code>activity_logs</code>	Admin action audit trail	Audit	<code>record_activity</code> , client logger, SQL triggers	Activity Logs p
23	<code>legal_documents</code>	Published Terms/Privacy versions	Legal	Migrations/admin SQL	Legal pages, s
24	<code>legal_consent</code>	One user's acceptance of a document version	Legal	Signup trigger <code>handle_new_user</code>	Audit
25	<code>photo_storage_settings</code>	Upload routing mode (Supabase or	Photo storage	Storage Monitoring controls	Upload helper

#	Table (schema)	Plain-language purpose	Business module	Written by	Read by
		Firebase fallback)			
26	photo_storage_events	Photo upload/cleanup/health log	Photo storage	Photo Edge Functions	Storage Monit
27	photo_cleanup_queue	Pending/finished photo deletion tasks	Photo storage	Scheduled and admin cleanup	Deletion work
28	manual_refund_reauth_attempts (private)	Failed admin re-authentication counter and lockout for manual refunds	Security	record-manual-refund via restricted RPCs	Same
29	paymongo_refund_recovery_jobs (private)	Queue that re-checks PayMongo for refunds made outside the app	Refund recovery	Payment/refund triggers	paymongo-re 5 min)

No table is unused. Every table has a live trigger, function, Edge Function, cron job, or page that writes or reads it. "No frontend reference" does not imply "unused"; for example, `order_status_events` is written only by a trigger, and `paymongo_refund_recovery_jobs` only by SQL and a worker.

Tables that no longer exist (do not document): `notification_delivery_attempts`, `email_usage_logs`, `email_activity_log`, `chat_faqs`, `chatbot_analytics`, `business_hours`. They appear only in old migrations or notes and are absent live.

4. Recommended core tables and reasons

4.1 Recommended main-chapter set (13 tables)

Present these, grouped by business flow and renumbered Tables 1–13:

New no.	Table	Class	Reason
1	Profiles	A	Every booking, message, alert, and feedback belongs to a profile; holds the customer/admin role.
2	Trips	A	The schedule, rate, and capacity that bookings are assigned to; records actual departure and hub arrival.
3	Orders	A	The booking itself: the central record of the whole business process.
4	Order_status_events	A	The tracking timeline shown to the public and customers.
5	Payment_transactions	A	The ledger of confirmed money received; the source of every balance.
6	Payment_refunds	A	Money returned; only successful refunds reduce money received.
7	Cancellation_settlements	A	The admin decision for money held on a cancelled booking. Missing from the current manuscript.
8	Announcements	A	Admin-published trip and company notices.
9	Notifications	A	In-app alerts customers receive when shipment or payment information changes.
10	Conversations	A	The customer's support thread and whether a human is needed.
11	Chat_messages	A	The messages inside the support thread.
12	Contact_inquiries	A	Visitor inquiries from the public contact form.
13	Customer_feedback	A	Post-delivery rating and review.

4.2 Supporting and technical tables

- **B (supporting, one paragraph each or an appendix table):** `company_information`, `activity_logs`, `payment_attempts`, `cancellation_settlement_history`, `email_subscriptions`, `legal_documents`, `legal_consent`s.
- **C (technical, summarize in one architecture paragraph):** `announcement_email_broadcasts`, `announcement_email_recipients`, `notification_delivery_jobs`, `user_device_tokens`, `photo_storage_settings`, `photo_storage_events`, `photo_cleanup_queue`, `manual_refund_reauth_attempts`, `paymongo_refund_recovery_jobs`.
- **D (uncertain):** none. All 29 tables are verified live.

Judgment call for the owner/adviser — `company_information`. It is configuration, but it directly drives business rules: the default rate and capacity copied into each new trip, the price fallback for a zero-rate trip, and the coverage list. The manuscript already has it (Table 4). Either keep it as a 14th main table (reasonable, since the chapter already contains it) or move it to the supporting paragraph. Both are defensible. Removing it from the chapter is **not** a reason to remove it from the system.

Relationship of `activity_logs`. The manuscript's current Table 1 is `Activity_logs`. Moving it to supporting text is a presentation choice only. It remains essential for auditing (rows kept 7 days).

No database change is recommended. No table should be dropped or merged to reduce the count.

5. Verified data dictionary for recommended core tables (LIVE)

Every live column is listed for each table below; **these are complete tables, not selected-field summaries**. Types, nullability, defaults, keys, and constraints were read live. The "Keys and rules" paragraph after each grid contains facts the three-column manuscript grid does not hold. Column order follows the live table.

5.1 Table 1. Profiles

Caption: Table 1. Profiles: This table stores the account information of each registered customer or administrator, including contact details, address, and system role.

Field Name	Data Type	Description
id	uuid	Unique identifier of the user, the same as the user's Supabase Authentication account.
name	varchar	Stores the full name of the user.
email	varchar	Stores the unique email address used for signing in.
phone	varchar	Stores the mobile number of the user.
address_lot_block	varchar	Stores the lot, block, or purok of the user's address.
address_street	varchar	Stores the street and subdivision of the user's address.
address_barangay	varchar	Stores the barangay of the user's address.
address_city	varchar	Stores the city or municipality of the user's address.
address_province	varchar	Stores the province of the user's address.
role	varchar	Indicates whether the account is a customer or an admin.
created_at	timestampz	Records the date and time the account was created.
updated_at	timestampz	Records the date and time the profile was last updated.
facebook_name	text	Stores the Facebook name of the user for contact purposes.
address_landmark	text	Stores a landmark near the user's address.
wants_announcements	bool	Indicates whether the user agreed to receive announcement emails.

Keys and rules: PK **id**, which is also an FK to **auth.users(id)** (cascade delete). **email** is unique. Required: **id, name, email, wants_announcements**. Defaults: **role = 'customer', wants_announcements = false**, timestamps **now()**. CHECK: **role** is **admin** or **customer**. Customers cannot change their own role (**guard_profile_write**). The profile is created by the signup trigger, which also refuses signup unless the current Terms and Privacy versions are accepted. Registration stores a **single full name**; the First/Last-name split applies only to booking sender/receiver fields.

5.2 Table 2. Trips

Caption: Table 2. Trips: This table stores each scheduled cargo trip, including its route, schedule, capacity, rate per kilogram, and the actual departure and arrival times at the destination hub.

Field Name	Data Type	Description
id	uuid	Unique identifier for each trip.
trip_number	varchar	Stores the unique trip code shown to users (for example, TRIP-20260820-979).
origin	varchar	Indicates the starting location of the trip.
destination	varchar	Indicates the destination of the trip.
departure_date	timestampz	Records the scheduled departure date of the trip.
arrival_date	timestampz	Records the planned arrival date of the trip.
capacity	int4	Stores the planned cargo capacity of the trip in kilograms.
price_per_kg	numeric	Stores the shipping rate per kilogram for bookings on this trip.
status	varchar	Indicates the current state of the trip: scheduled, in progress, arrived, completed, or cancelled.
notes	text	Stores optional remarks about the trip.
created_by	uuid	Stores the identifier of the admin who created the trip.
created_at	timestampz	Records the date and time the trip record was created.
updated_at	timestampz	Records the date and time the trip record was last updated.
departure_at	timestampz	Records the actual date and time the trip was started, set by the system.
estimated_arrival_at	timestampz	Stores the optional estimated arrival at the destination hub, entered when the trip starts.
arrived_at	timestampz	Records the actual date and time the trip arrived at the destination hub, set by the system.

Keys and rules: PK `id`; `trip_number` unique; `created_by` → `profiles(id)` (SET NULL). Required: `id`, `trip_number`, `origin`, `destination`, `departure_date`. Defaults: `status` = 'scheduled', `capacity` = 0, `price_per_kg` = 0. CHECKS: `status` list above; planned `arrival_date` must be after `departure_date`; `estimated_arrival_at` must be after actual `departure_at`; `arrived_at` cannot be before `departure_at`. A unique index allows **only one non-cancelled trip per route per Manila calendar day**. Status may only move scheduled → in_progress → arrived → completed, with cancellation allowed from any of the first three. `departure_at` and `arrived_at` can be set **only by the server** during Start/Arrive; they cannot be typed. `price_per_kg` is `numeric(10,2)`. The manuscript's `status` char is wrong: it is `varchar`.

5.3 Table 3. Orders

Caption: Table 3. Orders: This table stores each cargo booking, including the sender, receiver, and package details, its shipment status, and a summary of its charges and payments.

Field Name	Data Type	Description
<code>id</code>	<code>uuid</code>	Unique identifier for each booking.
<code>user_id</code>	<code>uuid</code>	Stores the identifier of the customer account that owns the booking.
<code>trip_id</code>	<code>uuid</code>	Stores the identifier of the trip the booking is assigned to.
<code>origin</code>	<code>varchar</code>	Indicates the starting location of the shipment.
<code>destination</code>	<code>varchar</code>	Indicates the destination of the shipment.
<code>tracking_number</code>	<code>varchar</code>	Stores the unique tracking number in the format CE-YYYYMMDD-NNNN.
<code>sender_name</code>	<code>varchar</code>	Stores the sender's full name, filled automatically from the first and last names.
<code>sender_phone</code>	<code>varchar</code>	Stores the sender's mobile number.
<code>sender_address</code>	<code>text</code>	Stores the sender's complete pickup address.
<code>receiver_name</code>	<code>varchar</code>	Stores the receiver's full name, filled automatically from the first and last names.
<code>receiver_phone</code>	<code>varchar</code>	Stores the receiver's mobile number.
<code>receiver_address</code>	<code>text</code>	Stores the receiver's complete delivery address.
<code>package_description</code>	<code>text</code>	Stores the customer's description of the items being shipped.
<code>actual_weight</code>	<code>numeric</code>	Records the actual weight in kilograms measured at pickup.
<code>shipping_cost</code>	<code>numeric</code>	Records the shipping fee before any discount, computed from the actual weight and the trip rate.
<code>payer_type</code>	<code>varchar</code>	Indicates whether the sender or the receiver pays for the shipment.
<code>payment_method</code>	<code>varchar</code>	Records the payment method used or chosen at pickup: cash, GCash, or pay later.
<code>payment_status</code>	<code>varchar</code>	Indicates whether the booking is unpaid, partially paid, or paid, derived from confirmed payments.
<code>amount_paid</code>	<code>numeric</code>	Records the total confirmed payments minus successful refunds.
<code>remaining_balance</code>	<code>numeric</code>	Records the amount still to be collected after the discount and confirmed payments.
<code>promised_payment_date</code>	<code>date</code>	Records the date the customer promised to pay the remaining balance.
<code>status</code>	<code>varchar</code>	Indicates the current shipment status of the booking, such as Pending, Assigned, or In Transit.
<code>notes</code>	<code>text</code>	Stores the customer's special instructions for the booking.
<code>created_at</code>	<code>timestampz</code>	Records the date and time the booking was created.
<code>updated_at</code>	<code>timestampz</code>	Records the date and time the booking was last updated.
<code>sender_facebook</code>	<code>text</code>	Stores the sender's Facebook name.
<code>sender_city</code>	<code>text</code>	Stores the sender's city or municipality.
<code>receiver_facebook</code>	<code>text</code>	Stores the receiver's Facebook name.
<code>receiver_city</code>	<code>text</code>	Stores the receiver's city or municipality.
<code>receiver_province</code>	<code>text</code>	Stores the receiver's province.
<code>sender_province</code>	<code>text</code>	Stores the sender's province, used to check service coverage.
<code>pickup_photos</code>	<code>jsonb</code>	Stores references to the photos taken at pickup.
<code>delivery_photos</code>	<code>jsonb</code>	Stores references to the proof-of-delivery photos.
<code>payment_reference</code>	<code>varchar</code>	Stores a payment reference number recorded for the booking.
<code>service_area_status</code>	<code>text</code>	Indicates whether the pickup location is within standard coverage, for review, approved, or rejected.
<code>service_area_remarks</code>	<code>text</code>	Stores the admin's remarks when reviewing an out-of-coverage booking.
<code>featured_on_website</code>	<code>bool</code>	Indicates whether the delivery is shown as a featured delivery on the public website.
<code>featured_title</code>	<code>text</code>	Stores the title used when the delivery is featured.

Field Name	Data Type	Description
featured_caption	text	Stores the caption used when the delivery is featured.
featured_image_type	text	Indicates which photo type (pickup or delivery) is used for the featured delivery.
featured_at	timestamptz	Records the date and time the delivery was featured.
reassignment_history	jsonb	Stores the history of trip reassignments and their reasons.
payment_preference	text	Stores the payment method the customer preferred during booking.
cancellation_details	jsonb	Stores the cancellation request: reason, previous status, timestamps, and admin review notes.
last_reminder_sent_at	timestamptz	Records when the last automatic payment reminder was emailed for this booking.
sender_barangay	text	Stores the sender's barangay.
sender_street	text	Stores the sender's street and subdivision.
sender_lot_block	text	Stores the sender's lot, block, or purok.
sender_landmark	text	Stores a landmark near the sender's address.
receiver_barangay	text	Stores the receiver's barangay.
receiver_street	text	Stores the receiver's street and subdivision.
receiver_lot_block	text	Stores the receiver's lot, block, or purok.
receiver_landmark	text	Stores a landmark near the receiver's address.
discount_amount	numeric	Records the shipping discount in pesos approved by the admin at pickup.
discount_reason	text	Indicates the reason for the discount: Regular customer, Negotiated price, or Other.
discount_notes	text	Stores the explanation required when the discount reason is Other.
discount_applied_by	uuid	Stores the identifier of the admin who applied the discount.
discount_applied_at	timestamptz	Records the date and time the discount was applied.
package_quantity	int4	Stores the number of boxes or parcels in the booking, from 1 to 50.
sender_first_name	text	Stores the sender's first name.
sender_last_name	text	Stores the sender's last name.
receiver_first_name	text	Stores the receiver's first name.
receiver_last_name	text	Stores the receiver's last name.

Keys and rules: PK `id`; `tracking_number` unique; `user_id` → `profiles(id)` (cascade); `trip_id` → `trips(id)` (SET NULL); `discount_applied_by` → `profiles(id)` (SET NULL). Required: `id`, `user_id`, `tracking_number`, sender/receiver name, phone and address, `discount_amount`, `package_quantity`, and the four first/last-name fields. Defaults: `status` = 'Pending', `payment_status` = 'unpaid', `shipping_cost` = 0, `amount_paid` = 0, `remaining_balance` = 0, `discount_amount` = 0, `package_quantity` = 1, `service_area_status` = 'standard', `payment_preference` = 'unspecified', photo and history arrays []. Money and weight are `numeric(10,2)`.

CHECKS (LIVE): `status` is one of Pending Review, Pending, Assigned, Picked Up, Pending Cancellation, In Transit, Arrived at Hub, Out for Delivery, Delivered, Cancelled; **a trip is required** for every status except Pending Review, Pending, Pending Cancellation, Cancelled; `actual_weight` is empty or between 0 (exclusive) and 10,000 kg; `amount_paid` and `remaining_balance` are never negative; discount is between 0 and `shipping_cost`, needs a reason when above 0, needs notes when the reason is Other; `package_quantity` 1–50; `payer_type` sender/receiver; `payment_method` cash/gcash/paylater; `payment_status` paid/partial/unpaid; sender/receiver first name cannot be blank (last names can be blank, for old records).

Server-enforced behavior (LIVE triggers): on insert the server generates the tracking number and forces weight, price, payments, photos, discount, and featured fields to empty/zero, so a customer **cannot** set their own price. Status becomes Assigned if a trip was chosen, otherwise Pending. Contact details lock once the booking is Out for Delivery. `sender_name/receiver_name` are kept for compatibility and are filled from the split names.

5.4 Table 4. Order_status_events

Caption: Table 4. Order_status_events: This table records every change in a booking's shipment status and is used to display the tracking timeline.

Field Name	Data Type	Description
id	uuid	Unique identifier for each status change.
order_id	uuid	Stores the identifier of the booking whose status changed.
status	varchar	Records the new status applied to the booking.
changed_at	timestamptz	Records the date and time of the status change.
changed_by	uuid	Stores the identifier of the user or admin who made the change.
note	text	Stores an optional remark about the change.

Keys and rules: PK `id`; `order_id` → `orders(id)` (cascade); `changed_by` → `profiles(id)` (SET NULL). Required: `id`, `order_id`, `status`, `changed_at` (default `now()`). Rows are written only by the trigger `log_order_status_event` when a booking is created or its status changes. A status may appear more than once (for example, after a declined cancellation request). The latest Delivered/Cancelled event is also used to decide when evidence photos expire (Section 10).

5.5 Table 5. Payment_transactions

Caption: Table 5. Payment_transactions: This table records every confirmed payment made for a booking, whether full or partial, in cash or GCash.

Field Name	Data Type	Description
<code>id</code>	<code>uuid</code>	Unique identifier for each payment.
<code>order_id</code>	<code>uuid</code>	Stores the identifier of the booking being paid.
<code>amount</code>	<code>numeric</code>	Records the amount received in this payment.
<code>payment_method</code>	<code>text</code>	Indicates how the payment was made: cash or GCash.
<code>transaction_reference</code>	<code>text</code>	Stores the GCash or PayMongo reference number of the payment.
<code>payment_status</code>	<code>text</code>	Indicates whether this payment settled the booking in full or in part.
<code>admin_id</code>	<code>uuid</code>	Stores the identifier of the admin who recorded the payment.
<code>admin_name</code>	<code>text</code>	Stores the name of the admin who recorded the payment.
<code>notes</code>	<code>text</code>	Stores remarks about the payment, such as "Initial pickup payment".
<code>created_at</code>	<code>timestampz</code>	Records the date and time the payment was recorded.
<code>payment_type</code>	<code>text</code>	Indicates when the payment was made, such as Initial Payment, Balance Settlement, or Additional Payment.
<code>payment_date</code>	<code>date</code>	Records the date the payment was actually received.
<code>receipt_url</code>	<code>text</code>	Stores the link to the uploaded proof-of-payment image.
<code>idempotency_key</code>	<code>uuid</code>	Stores a one-time key that prevents the same payment from being recorded twice.
<code>gcash_channel</code>	<code>text</code>	Indicates whether a GCash payment came through PayMongo or was recorded manually.
<code>transaction_reference_normalized</code>	<code>text</code>	Stores the reference number in a standard form to detect a reused GCash reference.

Keys and rules: PK `id`; `order_id` → `orders(id)` (cascade); `admin_id` → `profiles(id)` (SET NULL). Required: `id`, `order_id`, `amount`, `payment_method`, `payment_status`, `admin_name`, `created_at`. Unique (partial) indexes: `idempotency_key`; `transaction_reference`; normalized manual-GCash reference. So a retried request cannot create a second payment, and one GCash reference cannot be credited to two bookings. There is **no table-level** `amount > 0` **CHECK**; the positive-amount and "not more than what is owed" rules are enforced inside the recording RPCs (LIVE function definitions). Only this table counts as money received. A GCash checkout that is pending or failed lives in `payment_attempts`, never here. The live rows currently use `payment_status` values `paid` and `partial`. The manuscript's description ("paid or unpaid") should say **paid in full or partial**.

5.6 Table 6. Payment_refunds

Caption: Table 6. Payment_refunds: This table records every return of money for a booking, whether processed through PayMongo or recorded manually by the admin, together with its status.

Field Name	Data Type	Description
<code>id</code>	<code>uuid</code>	Unique identifier for each refund.
<code>refund_id</code>	<code>text</code>	Stores the refund identifier returned by PayMongo.
<code>idempotency_key</code>	<code>uuid</code>	Stores a one-time key that prevents duplicate refund requests.
<code>payment_transaction_id</code>	<code>uuid</code>	Stores the identifier of the payment being refunded.
<code>order_id</code>	<code>uuid</code>	Stores the identifier of the booking being refunded.
<code>payment_id</code>	<code>text</code>	Stores the PayMongo payment identifier of the original payment.
<code>amount</code>	<code>numeric</code>	Records the amount refunded.
<code>currency</code>	<code>text</code>	Stores the currency of the refund, always PHP.
<code>status</code>	<code>text</code>	Indicates the refund state: creating, pending, processing, succeeded, or failed.
<code>reason</code>	<code>text</code>	Indicates the reason category of the refund.
<code>notes</code>	<code>text</code>	Stores the admin's remarks about the refund.
<code>livemode</code>	<code>bool</code>	Indicates whether the refund was made in live or test mode.
<code>initiated_by</code>	<code>uuid</code>	Stores the identifier of the admin who started the refund.
<code>initiated_by_name</code>	<code>text</code>	Stores the name of the admin who started the refund.

Field Name	Data Type	Description
last_error	text	Records the most recent technical error for a failed refund.
last_event_id	text	Stores the identifier of the last PayMongo event processed for this refund.
provider_created_at	timestampz	Records when PayMongo created the refund.
provider_updated_at	timestampz	Records when PayMongo last updated the refund.
created_at	timestampz	Records the date and time the refund record was created.
updated_at	timestampz	Records the date and time the refund record was last updated.
outcome_uncertain	bool	Indicates that the system could not confirm whether PayMongo accepted the refund.
public_failure_reason	text	Stores a simple failure reason that is safe to show to the customer.
succeeded_at	timestampz	Records when the refund succeeded, used for report periods.
refund_channel	text	Indicates whether the refund was made through PayMongo or recorded manually.
return_method	text	Indicates whether a manual refund was returned in cash or GCash.
return_reference	text	Stores the GCash reference of a manual refund.
returned_at	timestampz	Records the date and time a manual refund was handed back.

Keys and rules: PK `id`; `payment_transaction_id` → `payment_transactions(id)` (cascade); `order_id` → `orders(id)` (cascade); `initiated_by` → `profiles(id)` (SET NULL). Unique partial indexes on `refund_id` and `idempotency_key`. Required: `id`, `payment_transaction_id`, `order_id`, `amount`, `currency`, `status`, `reason`, timestamps, `outcome_uncertain`, `refund_channel`. CHECKS: `amount > 0`; currency PHP; status list above; reason is duplicate, fraudulent, requested_by_customer, or others; a PayMongo refund must have `payment_id` and no `return_method`; a manual refund must have no PayMongo IDs and must have evidence (GCash reference of at least 4 characters with digits and no "@", or cash notes of at least 5 characters); PayMongo ID formats `pay_...` / `ref_...`. **Only succeeded refunds reduce money received.** The manuscript's field `livemod` is a typo for `livemode`, and its last five rows need descriptions.

5.7 Table 7. Cancellation_settlements (new to the manuscript)

Caption: Table 7. Cancellation_settlements: This table records the administrator's decision on how money already received for a cancelled booking is settled, either as a full refund or as an amount retained with the customer's agreement.

Field Name	Data Type	Description
order_id	uuid	Stores the identifier of the cancelled booking; each booking has at most one decision.
decision_type	text	Indicates the decision: full refund or retained amount.
agreed_retained_amount	numeric	Records the amount the business keeps with the customer's agreement; zero for a full refund.
customer_agreement_confirmed	bool	Indicates that the customer agreed to the retained amount.
customer_agreement_confirmed_at	timestampz	Records when the customer's agreement was confirmed.
internal_notes	text	Stores the admin's private notes about the decision.
decided_by	uuid	Stores the identifier of the admin who recorded the decision.
decided_at	timestampz	Records when the decision was first recorded.
updated_at	timestampz	Records when the decision was last amended.
version	int4	Counts how many times the decision has been recorded or amended.
last_idempotency_key	uuid	Stores a one-time key that prevents the same decision from being saved twice.

Keys and rules: PK and FK `order_id` → `orders(id)` (cascade) — a **one-to-one** relationship. `decided_by` → `profiles(id)` (RESTRICT). `last_idempotency_key` unique. CHECKS: `decision_type` is `full_refund` or `retained_fee`; a full refund has retained amount 0 and no agreement flag; a retained amount must be above 0 **and** have confirmed customer agreement with a timestamp; notes ≤ 1,000 characters; version ≥ 1. The recording RPC also rejects a retained amount greater than the money actually held or the booking's final charge. Each change is copied to `cancellation_settlement_history`. **This table records a decision only. It does not itself collect or return money.**

5.8 Table 8. Announcements

Caption: Table 8. Announcements: This table stores the announcements published by the administrator for customers, visitors, or email subscribers.

Field Name	Data Type	Description
id	uuid	Unique identifier for each announcement.
title	varchar	Stores the headline of the announcement.
content	text	Stores the full text of the announcement.
author_id	uuid	Stores the identifier of the admin who published the announcement.

Field Name	Data Type	Description
is_active	bool	Indicates whether the announcement is currently visible.
created_at	timestampz	Records the date and time the announcement was created.
updated_at	timestampz	Records the date and time the announcement was last updated.
comments	jsonb	Stores the customer comments posted on the announcement.
send_email	bool	Indicates whether the admin requested that the announcement be emailed to subscribers.
emailed_at	timestampz	Records when the email broadcast was completed.
cta_label	text	Stores the optional label of the announcement's action button.
cta_url	text	Stores the optional secure (https) link of the announcement's action button.
audience	text	Indicates whether the announcement is public or sent by email only.

Keys and rules: PK `id`; `author_id` → `profiles(id)` (SET NULL). Required: `id`, `title` (max 200 characters), `content`, `send_email`, `audience`. Defaults: `is_active` = `true`, `send_email` = `false`, `audience` = `'public'`, `comments` = `[]`. CHECKs: `audience` is `public` or `email_only`; `cta_url` must start with `https://`. `send_email` = `true` records the admin's request only. Actual delivery is tracked in the broadcast/recipient job tables and is **not** guaranteed by this flag.

5.9 Table 9. Notifications

Caption: Table 9. Notifications: This table stores the in-app notifications sent to a customer or administrator when bookings, trips, payments, announcements, or messages change.

Field Name	Data Type	Description
id	uuid	Unique identifier for each notification.
user_id	uuid	Stores the identifier of the user who receives the notification.
title	varchar	Stores the short title of the notification.
message	text	Stores the content of the notification.
type	varchar	Indicates the category, such as order update, trip update, payment update, or announcement.
reference_id	uuid	Stores the identifier of the related record, such as a booking, trip, or announcement.
is_read	bool	Indicates whether the user has opened the notification.
created_at	timestampz	Records the date and time the notification was created.
payment_transaction_id	uuid	Stores the identifier of the related payment, for payment notifications.
payment_refund_id	uuid	Stores the identifier of the related refund, for refund notifications.

Keys and rules: PK `id`; `user_id` → `profiles(id)` (cascade); `payment_transaction_id` and `payment_refund_id` → their tables (SET NULL). Required: `id`, `user_id`, `title`, `message`. Defaults: `type` = `'general'`, `is_read` = `false`. CHECK: `type` is one of `order_update`, `trip_update`, `announcement`, `general`, `inquiry`, `feedback`, `chat_message`, `system_alert`, `payment_update`. Unique partial indexes stop the same alert being created twice (e.g., one alert per payment, per refund, per announcement per user). Notifications are created by database triggers, not by the browser. Phone/browser **push** delivery is a separate queue and may be skipped when a user has no registered device (live: 230 of 280 jobs skipped for that reason).

5.10 Table 10. Conversations

Caption: Table 10. Conversations: This table stores the support conversation of each customer and indicates whether the automated assistant or an administrator is handling it.

Field Name	Data Type	Description
id	uuid	Unique identifier for each conversation.
customer_id	uuid	Stores the identifier of the customer who owns the conversation.
created_at	timestampz	Records the date and time the conversation started.
status	text	Indicates the state: assistant active, waiting for admin, waiting for customer, or resolved.
escalated	bool	Indicates that the customer was handed over to an administrator.
first_response_at	timestampz	Records when an administrator first replied.
last_customer_message_at	timestampz	Records when the customer last sent a message.
resolved_at	timestampz	Records when the conversation was marked resolved.
bot_resolved	bool	Indicates whether the customer confirmed that the assistant answered the concern.

Keys and rules: PK `id`; `customer_id` → `profiles(id)` (cascade) and **unique**. Each customer has **exactly one** conversation that is reused, not a new thread per concern. Required: `id`, `customer_id`, `escalated`. Defaults: `status` = `'bot_active'`, `escalated` = `false`. CHECK: `status` is `bot_active`, `waiting`,

waiting_customer, or resolved. State changes are made by a trigger when messages arrive: a customer message while an admin is involved → **waiting**; an admin reply → **waiting_customer**; a customer message after **resolved** → back to **bot_active** with **escalated** reset (a fresh assistant session). A daily job resolves **waiting_customer** threads idle for 7 days.

5.11 Table 11. Chat_messages

Caption: Table 11. Chat_messages: This table stores each message sent in a support conversation by the customer, the automated assistant, or an administrator.

Field Name	Data Type	Description
id	uuid	Unique identifier for each message.
conversation_id	uuid	Stores the identifier of the conversation the message belongs to.
sender_id	uuid	Stores the identifier of the account that sent the message.
sender_role	varchar	Indicates who sent the message: customer, bot, or admin.
message	text	Stores the text of the message.
is_read	bool	Indicates whether the recipient has read the message.
created_at	timestampz	Records the date and time the message was sent.

Keys and rules: PK **id**; **conversation_id** → **conversations(id)** (cascade); **sender_id** → **profiles(id)** (cascade). Required: **id**, **conversation_id**, **sender_id**, **sender_role**, **message**. CHECK: message 1–1,000 characters after trimming. The insert trigger sets **sender_id** to the signed-in user and **sender_role** to the user's real role, **except** that a customer may insert a message labelled **bot**. Assistant replies are generated in the customer's browser and saved under the customer's own ID with role **bot**. The manuscript should say the assistant's replies are "recorded as assistant messages", not that a server-side bot sent them. See Section 12, issue 1.

5.12 Table 12. Contact_inquiries

Caption: Table 12. Contact_inquiries: This table stores the inquiries submitted by visitors through the public contact form and tracks how the administrator handled them.

Field Name	Data Type	Description
id	uuid	Unique identifier for each inquiry.
name	text	Stores the name of the visitor who sent the inquiry.
phone	text	Stores the phone number entered by the visitor.
message	text	Stores the visitor's question or concern.
status	text	Indicates whether the inquiry is new, read, or resolved.
created_at	timestampz	Records the date and time the inquiry was received.
contact_phone	text	Stores an alternative contact number for the inquiry.
contact_email	text	Stores the email address entered by the visitor.
assigned_admin_id	uuid	Stores the identifier of the admin assigned to handle the inquiry.
first_response_at	timestampz	Records when the inquiry was first opened or answered by an admin.
resolved_at	timestampz	Records when the inquiry was marked resolved.
push_dispatched_at	timestampz	Records when the new-inquiry alert was sent to the admins.
push_dispatch_started_at	timestampz	Records when the system began sending the new-inquiry alert.
push_dispatch_claim_id	uuid	Stores a temporary job identifier that prevents the alert from being sent twice.
ip	text	Stores the visitor's IP address, used to limit repeated submissions.
wants_announcements	bool	Indicates whether the visitor asked to receive announcement emails.

Keys and rules: PK **id**; **assigned_admin_id** → **profiles(id)** (SET NULL). Required: **id**, **name**, **phone**, **message**, **status**, **created_at**, **wants_announcements**. CHECKS: name 2–100 characters, phone 6–100, message 10–2,000; status **new**, **read**, or **resolved**. A rate-limit trigger blocks rapid repeat submissions. The manuscript's description of **push_dispatch_claim_id** ("identifier of the admin who claimed...") is wrong. It is a job identifier, not an admin.

5.13 Table 13. Customer_feedback

Caption: Table 13. Customer_feedback: This table stores the rating and review submitted by a customer for a delivered booking.

Field Name	Data Type	Description
id	uuid	Unique identifier for each feedback entry.
order_id	uuid	Stores the identifier of the delivered booking being reviewed.
customer_id	uuid	Stores the identifier of the customer who gave the feedback.

Field Name	Data Type	Description
rating	int4	Records the star rating from 1 to 5.
message	text	Stores the customer's written review.
is_hidden	bool	Indicates whether the admin has hidden the review from public display.
created_at	timestampz	Records the date and time the feedback was submitted.

Keys and rules: PK `id`; `order_id` → `orders(id)` (cascade) and **unique** — one review per booking; `customer_id` → `profiles(id)` (cascade). All columns are required. Defaults: `is_hidden = false`. CHECKs: rating 1–5; message 1–2,000 characters. Hiding and restoring is logged in `activity_logs`.

5.14 Relationships and cardinality among the core tables (LIVE foreign keys)

Parent	Child	Cardinality	On parent delete
auth user	Profiles	1 : 1	cascade
Profiles	Orders (<code>user_id</code>)	1 : many	cascade
Trips	Orders (<code>trip_id</code>)	1 : 0..many (booking may be unassigned)	set null
Orders	Order_status_events	1 : many	cascade
Orders	Payment_transactions	1 : many	cascade
Payment_transactions	Payment_refunds	1 : many	cascade
Orders	Payment_refunds	1 : many	cascade
Orders	Cancellation_settlements	1 : 0..1	cascade
Orders	Customer_feedback	1 : 0..1	cascade
Profiles	Customer_feedback	1 : many	cascade
Profiles	Conversations	1 : 0..1	cascade
Conversations	Chat_messages	1 : many	cascade
Profiles	Chat_messages (<code>sender_id</code>)	1 : many	cascade
Profiles	Notifications	1 : many	cascade
Payment_transactions / Payment_refunds	Notifications	1 : 0..1 each	set null
Profiles	Announcements (<code>author_id</code>)	1 : many	set null
Profiles	Contact_inquiries (<code>assigned_admin_id</code>)	1 : many	set null
Profiles	Trips (<code>created_by</code>)	1 : many	set null

`package_quantity` does **not** create child parcel records. All boxes in a booking share one status, one weight, and one fee.

6. Table and relationship differences from Chapter 2

Manuscript table (PDF page)	Differences from the live database	Action
1 Activity_logs (p. 51)	Columns match. Missing fact: rows are deleted after 7 days.	Move to supporting text; mention 7-day retention.
2 Announcements (p. 52)	Missing <code>cta_label</code> , <code>cta_url</code> , <code>audience</code> . <code>send_email</code> is a request, not proof of delivery.	Replace with Section 5.8.
3 Chat_messages (p. 52)	Missing <code>is_read</code> . <code>sender_role</code> values are customer/bot/admin; bot messages are saved under the customer's ID.	Replace with Section 5.11.
4 Company_information (p. 53)	Columns match live (21), but <code>default_capacity</code> , <code>features</code> , <code>coverage</code> have no descriptions. <code>messenger</code> no longer exists (removed 20 Sep).	Complete descriptions; keep as table 14 or supporting (Section 4.2).
5 Contact_inquiries (p. 54)	Columns match. <code>created_at</code> description contains a stray "timestamp with time zone ". <code>push_dispatch_claim_id</code> is described wrongly as an admin.	Replace with Section 5.12.
6 Conversations (p. 55)	Columns match. <code>escalated</code> means handed to an admin, not "urgent". Missing the rule of one conversation per customer .	Replace with Section 5.10.
7 Customer_feedback (p. 55)	Missing <code>is_hidden</code> , <code>created_at</code> . Missing the rule of one review per booking.	Replace with Section 5.13.

Manuscript table (PDF page)	Differences from the live database	Action
8 Notifications (p. 56)	Missing <code>title</code> , <code>payment_transaction_id</code> , <code>payment_refund_id</code> .	Replace with Section 5.9.
9 Order_status_events (p. 56)	Columns match.	Minor wording (Section 5.4).
10 Orders (pp. 57–59)	Manuscript lists 44 fields; live has 63 . Missing: <code>sender_name</code> , <code>receiver_name</code> , <code>featured_at</code> , <code>last_reminder_sent_at</code> , <code>discount_notes</code> , and all 8 structured address parts (<code>sender_barangay</code> , <code>sender_street</code> , <code>sender_lot_block</code> , <code>sender_landmark</code> , and the receiver equivalents). Name fields are <code>text</code> , not <code>varchar</code> . <code>discount_reason</code> is a fixed choice, not "detailed notes" (that is <code>discount_notes</code>). <code>amount_paid</code> is net of successful refunds. <code>payment_status</code> includes partial. <code>timestampz</code> is a typo. <code>package_quantity</code> counts boxes in one booking.	Replace with Section 5.3.
11 Payment_transactions (p. 59)	Manuscript lists 9 fields; live has 16. Missing: <code>notes</code> , <code>created_at</code> , <code>payment_type</code> , <code>payment_date</code> , <code>idempotency_key</code> , <code>gcash_channel</code> , <code>transaction_reference_normalized</code> . Records only confirmed payments.	Replace with Section 5.5.
12 Profiles (p. 60)	Missing <code>role</code> , <code>created_at</code> , <code>updated_at</code> , <code>wants_announcements</code> . The caption mentions "system roles" but the role column is absent.	Replace with Section 5.1.
13 Trips (pp. 60–61)	Missing <code>notes</code> , <code>created_by</code> , <code>created_at</code> , <code>updated_at</code> , <code>estimated_arrival_at</code> , <code>arrived_at</code> . <code>status</code> is <code>varchar</code> , not <code>char</code> . <code>arrival_date</code> is the planned arrival, not "expected or actual".	Replace with Section 5.2.
14 Payment_refunds (pp. 61–62)	Columns match live (27). <code>livemod</code> → <code>livemode</code> ; last five fields lack descriptions.	Replace with Section 5.6.
Missing	<code>cancellation_settlements</code> — a core business table that the cancellation process depends on.	Add as new Table 7.
Missing (supporting)	15 other tables (Section 3).	Summarize in one paragraph or appendix table.

Relationship corrections: the manuscript never states cardinality. If an ERD is added, use Section 5.14. Two cardinalities are commonly drawn wrong: Profiles–Conversations is **one-to-one**, and Orders–Customer_feedback is **one-to-zero-or-one**.

7. Complete module/process coverage matrix

Every route in `src/App.jsx` is mapped below. `ProtectedRoute` checks the user's role for convenience only; the real authorization is enforced by database RLS policies and functions.

Route(s)	Process no. (Section 8)	Actor & trigger	Inputs & preconditions	Backend authorization/validation & data	Success / blocked cases	Backgrou
<code>/about</code>	1	Visitor opens page or submits contact form	Name, mobile, email, message, optional announcement opt-in	<code>submit-inquiry</code> Edge Function; length CHECKS; IP/phone rate limit; writes <code>contact_inquiries</code> , optional <code>email_subscriptions</code>	"Message sent! We will contact you soon." / validation or rate-limit message	Admin not <code>notify_a</code>
<code>/track</code> , <code>/customer/track</code>	2	Anyone enters a tracking number	Tracking number	<code>track_order_public</code> / <code>get_public_order_events</code> (security-definer, name-masked) read <code>orders</code> , <code>trips</code> , <code>order_status_events</code>	Status, route, timeline, progress / "Shipment Not Found" / temporary cooldown after rapid searches	Auto-refre
<code>/schedules</code> , <code>/customer/trips</code>	3	Visitor or customer views trips	none	Public trip list with load (<code>get_trips_load</code>)	Visitor selecting a trip: toast "Login to book the schedule or inquire" → login; customer: booking form with route/trip preselected	—
<code>/login</code>	4	User signs in	Email, password	Supabase Auth; profile role read	Role-based redirect / error message	Activity lo
<code>/register</code>	5	Visitor registers	Account step (full name, Facebook name, email, mobile,	Supabase Auth signup; <code>handle_new_user</code> creates profile and <code>legal_consent</code> s, refuses signup without consent	"Account Created!" / field or signup errors	<code>email_su</code>

Route(s)	Process no. (Section 8)	Actor & trigger	Inputs & preconditions	Backend authorization/validation & data	Success / blocked cases	Background
			password x2); address step; Terms/Privacy consent; optional email opt-in			
/forgot-password, /reset-password	6	User requests a reset	Email; later new password x2	Supabase Auth recovery	"Check Your Inbox" with resend countdown / generic send failure / "Link Expired or Invalid" / "Password Updated!"	Reset email
/customer	7	Customer opens home	—	Own orders (RLS), announcements, next trip	Summary counts, next trip card ("Book Cargo for This Trip"), active shipments, announcements	—
/customer/book	8	Customer books	Route, optional trip, sender and receiver (first/last name, mobile, Facebook, structured address), package description, who pays, optional payment preference, notes	Insert into orders ; triggers force price/weight/payments to empty, generate tracking number, set status Pending/Assigned, flag out-of-coverage province as for_review , check trip still open (scheduled, not past its Manila day) and capacity, validate names	Success page with tracking number / step validation errors / "This trip is full..." / "...no longer accepting bookings"	Admin "New customer"
/customer/orders, /customer/orders/:id, /payment/return	9	Customer views and acts on a booking	Contact edits; payment amount; cancellation reason; rating and review	update_order_contact_details (locked from Out for Delivery); paymongo-create-payment → payment_attempts ; webhook / verify-payment-return → payment_transactions ; request_order_cancellation (only Pending Review/Pending/Assigned; reason 5–500 chars); feedback insert (Delivered only, one per booking)	"Payment confirmed!" / "Payment will become available once your shipment has been picked up and weighed" / pending or failed message / "Thank you! Your feedback has been submitted."	Payment, I
/customer/notifications	10	Customer manages alerts	Selection, "Mark all read", "Clear all"	Own notifications rows (RLS)	Opens linked booking/trip/announcement	—
/customer/support	11	Customer uses support	Menu choice or typed message; "Talk to an admin"	supportChatEngine.js (browser) reads the customer's own data; messages to chat_messages ; trigger updates conversations	Menu/rule answer / handoff "Connecting you to an admin..."	Admin not (notify_
/customer/payments, /customer/payment-methods (redirect)	12	Customer views payments	Month filter, selection	get_payment_transaction_history , get_payment_refund_history , get_payment_attempt_history (owner-restricted)	Outstanding balance, total paid, payment list, details	—
/customer/profile, /customer/personal-info, /customer/change-email, /customer/change-password, /customer/help-guidelines, /customer/about-version, /faq	13	Customer manages account	Profile fields; passwords; new email + password	profiles update (role locked); Supabase Auth	"Profile updated successfully!" / "Check your new inbox" / errors	Email-cha Supabase

Route(s)	Process no. (Section 8)	Actor & trigger	Inputs & preconditions	Backend authorization/validation & data	Success / blocked cases	Backgrou
/admin	14	Admin opens dashboard	—	Admin-only counts and charts	Pending, awaiting departure, in transit, active trips; capacity gauge; status chart; recent orders	—
/admin/orders, /admin/orders/:id	15, 28, 29	Admin processes bookings	Trip; weight, payer, payment, discount, photos; delivery photos and balance; cancellation decision; coverage decision; reassignment reason; label count	record_pickup_payment, record_delivery_payment, record_additional_payment, reassign_trip, review_order_cancellation, settlement and refund functions; guard_order_update (capacity, discount, dispatch gate, contact lock)	Picked Up / Delivered / blocked with reason (see Section 8, process 15)	Customer
/admin/create-booking	16	Admin creates walk-in	Same as booking	Same insert triggers	Tracking number shown	Same as b
/admin/trips, /admin/trips/create, /admin/trips/:id	17	Admin manages trips	Route, departure and planned arrival; optional email; reschedule dates + reason; optional ETA at start	Insert trip (rate/capacity copied from Company Information); browser auto-assigns matching Pending bookings; reschedule_trip; guard_trip_status_transition and validate_trip_status_transition	Trip saved / start blocked (early, overdue, pickups missing, pending cancellations) / completion blocked (undelivered or unpaid)	Trip notific email broe
/admin/customers, /admin/customers/:id	18	Admin looks up customers	Search text, province	get_admin_customers	Profile and booking history	—
/admin/sales, /admin/reports	19	Admin generates report	Month	Both routes render PerTripSalesPage via SalesReportsPage; reads trips in the month, their current bookings, payment/refund activity, settlements	Monthly grand total and per-trip details / "The trip report could not be loaded."	"Print Rep
/admin/announcements	20	Admin publishes	Category, title (≤100 in form), content (≤1,500 in form), optional email, audience/link	Insert announcements; triggers notify customers; email broadcast queue	Published / "Please enter a title — this is what customers see first" / "Please write the announcement content."	In-app not
/admin/inbox	21	Admin replies	Reply text; resolve action	chat_messages insert (role forced to admin); trigger sets waiting_customer	Thread updated	Customer
/admin/contact-inquiries	22	Admin handles inquiries	Status, assignment	Admin-only update; resolve-ownership guard	Status saved	—
/admin/feedback	23	Admin moderates	Filter, Hide/Restore	customer_feedback.is_hidden update	Visibility changed	Activity lo
/admin/activity-logs	24	Admin audits	Search, module, hide sign-ins	Read activity_logs (last 7 days only)	Filtered list; "Export CSV"	—
/admin/company-info	25	Admin edits settings	Tab fields	Singleton update	"Changes saved successfully"	Activity lo
/admin/storage-monitoring	26	Admin checks storage	Optional photo selection for deletion	Storage summary RPCs; delete_evidence_photos; delete-storage-photos	Health status; permanent deletion of selected eligible photos	photo_st

Route(s)	Process no. (Section 8)	Actor & trigger	Inputs & preconditions	Backend authorization/validation & data	Success / blocked cases	Backgrou
/admin/profile, /admin/change-email, /admin/change-password	27	Admin manages own account	Email/password	Supabase Auth	Success/error	—
/terms, /privacy, /unsubscribe,*	1 (note), 30	Visitor	Unsubscribe token	unsubscribe-announcements Edge Function updates email_subscriptions	Legal text; unsubscribed; 404 page	—
(no route) scheduled jobs	30	pg_cron	—	See Section 10	—	Reminders cleanup

Features that do not exist (do not describe them): automated route sequencing or optimization; separate tracking per box/parcel; receiver identity verification; automatic delivery confirmation by QR scan; an in-app QR scanner; expense, cost, or profit calculation; tip recording (overpayment is rejected, not kept as a tip); generative AI or natural-language model chat; a "PDF export" button on the current report page.

8. Process specifications in the manuscript's existing format

These are replacement drafts for PDF pp. 19–47, written in the manuscript's own style. The numbering 1–27 is kept; 28–30 are proposed additions. Quoted button labels and messages were checked against the current source code (they match or were corrected to the actual wording).

```
1. Home and About Us
Begin
  Display Company Overview, Features, Coverage Areas, Trip Schedules, and Gallery
  If Visitor selects "Sign In"
    Redirect to Login Page
  End If
  If Visitor scrolls to Contact Us
    Display Contact Form
    Visitor enters Name, Mobile Number, Email Address, and Message
    Visitor optionally checks "I want to receive email updates regarding trip schedules, promos, and announcements."
    Click "Send Message"
    If input is valid and the submission limit is not exceeded
      System records the inquiry and notifies the administrators
      Display "Message sent! We will contact you soon."
    Else If submissions are sent too frequently
      Display a temporary limit message
    Else
      Display field validation errors
    End If
  End If
End
```

```
2. Track Package Process
Begin
  Display Tracking Search Box
  User enters Tracking Number
  Click "Track"
  If tracking number is found
    Display Current Status, Route, and Package Details with masked names
    Display Shipment Journey Timeline and progress bar
    System automatically refreshes the tracking status
  Else
    Display "Shipment Not Found" message
  End If
  If repeated searches occur too quickly
    Display a temporary cooldown message
  End If
End
```

```
3. Trip Schedules Process
Begin
  Display list of scheduled trips with Route, Departure Date, and Available Capacity
  If User is not logged in and selects a trip
    Display "Login to book the schedule or inquire" message
    Redirect to Login Page
```

```

Else If Customer selects a trip
    Redirect to Book Shipment with the route and trip pre-selected
End If
End

```

```

4. Login Process
Begin
    Display Login Form
    User enters Email Address and Password
    Click "Sign In"
    If credentials are valid
        If account role is Administrator
            Redirect to Admin Dashboard
        Else
            Redirect to Customer Home
        End If
    Else
        Display "Incorrect password or email" message
    End If
End

```

```

5. Customer Registration Process
Begin
    Display Registration Form – Account Step
    Customer enters Full Name, Facebook Name, Email Address, Mobile Number, Password, and Confirm Password
    Click "Continue to Address"
    If account input is valid
        Display Registration Form – Address Step
        Customer selects Province and City/Municipality
        Customer enters Barangay, Street, Lot/Block/Purok, and Landmark
        Customer checks "I have read and agree to the Terms of Service and Privacy Policy."
        Customer optionally checks "I want to receive email updates regarding trip schedules, promos, and
announcements. (Optional)"
        Click "Create Account"
        If registration is successful
            System saves the account, profile, and the accepted Terms and Privacy Policy versions
            Display "Account Created!" confirmation
            Redirect to Customer Home
        Else
            Display error message
        End If
    Else
        Display field validation errors
    End If
End

```

```

6. Password Recovery Process
Begin
    Display Email Input Form
    User enters Registered Email Address
    Click "Send Reset Link"
    If the reset request is accepted
        System sends a password reset link to the email address
        Display "Check Your Inbox" confirmation
        If user clicks "Resend Email" before 60 seconds pass
            Display cooldown message
        End If
    Else
        Display "We couldn't send the reset link. Please try again in a few minutes, or contact support."
    End If
    User opens the emailed link
    System verifies the reset link
    If link is invalid or expired
        Display "Link Expired or Invalid" message
        User may request a new link
    Else
        Display New Password and Confirm Password fields
        User enters New Password and Confirm Password
        Click "Update Password"
        If password meets requirements and both entries match
            System updates the account password
            Display "Password Updated!" confirmation
            Redirect to Login Page
        Else

```

```

        Display validation error
    End If
End If
End

```

Correction: the system does **not** display "Email not found". It does not reveal whether an email is registered.

```

7. Customer Home Process
Begin
    Display greeting and booking summary (Total, Active, Delivered)
    System loads latest bookings, announcements, and the next available trip
    If a next available trip exists
        Display trip route, dates, and available capacity
        If Customer clicks "Book Cargo for This Trip"
            Redirect to Book Shipment with the route and trip pre-selected
        End If
    End If
    If announcements exist
        Display latest announcements with option to view and comment
    End If
    If active shipments exist
        Display active shipment cards
        If Customer selects a shipment
            Redirect to Order Detail
        End If
    End If
    If Customer enters a tracking number in the search box
        Click "Track"
        Redirect to Track Package
    End If
End

```

```

8. Book Shipment Process
Begin
    Display Route Step
    Customer selects Route
    Customer optionally selects a specific Trip
    Click "Continue"
    Display Sender Details Step
    Customer enters Sender First Name, Last Name, Mobile Number, Facebook Name, Province, City/Municipality,
    Barangay, Street, Lot/Block/Purok, and Landmark
    Click "Continue"
    Display Receiver Details Step
    Customer enters the same details for the Receiver
    Click "Continue"
    Display Package Details Step
    Customer describes the package, selects Who Pays, and optionally states a Payment Preference and Special
    Instructions
    Click "Review Booking"
    Display Review & Confirm Step
    Customer reviews Route, Sender, Receiver, and Package details
    Click "Confirm Booking"
    If all required information is valid and the selected trip is still open and not full
        System saves the booking without weight or price and generates a Tracking Number
        If a trip was selected
            System marks the booking as Assigned
        Else
            System marks the booking as Pending
        End If
        If the sender's province is outside the standard coverage area
            System flags the booking For Review by the administrator
        End If
        Display Success Page with Tracking Number
    Else
        Display validation error and return to the incomplete step
    End If
End

```

Note: standard coverage provinces (LIVE) are Bohol, Metro Manila, Cavite, Batangas, Laguna, and Bulacan.

```

9. Bookings Process
Begin
    Display list of the Customer's bookings with status filters
    Customer searches or selects a booking

```

```

Redirect to Order Detail
Display Tracking Timeline, Sender/Receiver, Package Details, and Shipment Proof photos
Display Payment Details (Shipping Cost, Discount, Paid, Balance, Method, Status)
If the booking is not yet Out for Delivery and Customer edits contact details
    System saves the corrected contact details
End If
If a balance is owed and the shipment has been weighed and picked up
    Customer enters an amount to pay
    Click "Pay [amount] with GCash"
    System opens the GCash checkout
    If payment is confirmed by the payment provider
        Display "Payment confirmed!" and update the balance
    Else
        Display payment failed/still-processing message with a retry option
    End If
Else If the shipment has not yet been picked up and weighed
    Display "Payment will become available once your shipment has been picked up and weighed"
End If
If booking status is Pending Review, Pending, or Assigned
    Click "Request Cancellation"
    Customer enters a reason
    System records the cancellation request and marks the booking as Pending Cancellation
    Display confirmation message
End If
If booking status is "Delivered" and feedback has not yet been given
    Display "How was your delivery?" review prompt
    Customer selects a Star Rating and enters a message
    Click "Submit Feedback"
    If input is valid
        System records the feedback
        Display "Thank you! Your feedback has been submitted."
    Else
        Display validation error
    End If
End If
End

```

10. Notifications Process

```

Begin
    Display notifications grouped by date, each marked read or unread
    If Customer selects a notification
        If notification relates to a booking or trip
            Redirect to the related Bookings or Trips screen
        Else If notification is an announcement
            Display the full announcement and its comments
        End If
    End If
    If Customer clicks "Mark all read"
        System marks all notifications as read
    End If
    If Customer clicks "Clear all"
        Display confirmation prompt
        If confirmed
            System removes all notifications
        End If
    End If
End

```

11. Chat Support Process

```

Begin
    Display chat conversation window
    Display menu: "My bookings", "Payment and refunds", "How to book", "Shipping information", "Talk to an admin"
    Customer selects a menu option or types a message in English, Tagalog, or Bisaya
    If Customer selects "My bookings"
        Display the Customer's bookings
        Customer selects a booking
        Display its status, payment details, or trip details
    Else If the message or option matches a prepared topic
        The automated assistant displays the prepared answer and follow-up options
    Else If Customer selects "Talk to an admin" or the message requires an administrator
        System forwards the conversation to the administrator
        Display "Connecting you to an admin..." message
    Else
        Display the menu again with the option to talk to an admin
    End If
    If the conversation was resolved and the Customer sends a new message

```

```

        A new automated assistant session begins
    End If
End

```

Correction: the current system does **not** reopen a resolved conversation with the same administrator. The earlier 12-hour reopen rule was removed (migration 20260920214200). No administrator is assigned to a conversation.

```

12. Payment History Process
Begin
    Display Outstanding Balance, Total Paid, and Active Orders summary
    If open balances exist
        Display list of unpaid or partially paid bookings
        If Customer selects a booking
            Redirect to Order Detail
        End If
    End If
    If payment records exist
        Display Recent Payments list with Date, Order, Type, Amount, Method, and Status
        Display refunds separately; only successful refunds reduce the total paid
        Customer may filter by month
        If Customer selects a payment
            Display payment detail (reference number, recorded by, notes)
        End If
    End If
End

```

```

13. Profile Management Process
Begin
    Display account summary and profile completion status
    If Customer selects "Personal Info & Addresses"
        Display current Full Name, Facebook Name, Mobile Number, and Address
        Customer edits fields
        Click "Save Changes"
        If input is valid
            System updates the profile
            Display "Profile updated successfully!" message
        Else
            Display validation error
        End If
    Else If Customer selects "Change Password"
        Customer enters Current Password, New Password, and Confirm Password
        Click "Update Password"
        If current password is correct and new password meets requirements
            Display success message
        Else
            Display error message
        End If
    Else If Customer selects "Change Email"
        Customer enters New Email Address and Current Password
        Click "Update Email"
        If input is valid
            Display "Check your new inbox" message
        Else
            Display error message
        End If
    Else If Customer selects "Help & Guidelines" or "About & Version"
        Display the corresponding reference content
    End If
    If Customer clicks "Sign Out"
        Display confirmation prompt
        If confirmed
            End session and redirect to Login Page
        End If
    End If
End

```

```

14. Admin Dashboard Process
Begin
    Display Pending Bookings, Awaiting Departure, In Transit, and Active Trips counts
    Display current trip capacity gauge
    Display order status distribution chart
    Display Recent Orders list
    If Admin selects a recent order
        Redirect to Bookings Management
    End If
End

```

```
End If
End
```

15. Bookings Management Process

Begin

Display list of bookings with status tabs (All, Action Needed, Pending, Active, Completed, Cancelled)

Admin searches or selects a status tab

If Admin selects a booking

Display booking details, tracking timeline, and payment summary

If booking was flagged For Review because it is outside standard coverage

Admin reviews the request

Click "Approve Request" or "Reject Request"

If rejected

System cancels the booking with the Admin's remarks

End If

End If

If booking is unassigned

Admin selects an available Trip

Click "Assign to Trip"

System marks the booking as Assigned

End If

If next step is pickup

Click "Process Pickup"

Admin enters the Actual Weight

System displays the computed shipping cost (weight × rate per kilo)

Admin optionally enters a Discount with a reason

Admin selects Who Pays (Sender now, or Receiver on delivery)

If Sender pays now

Admin selects Full Payment or Pay Later, enters amount and payment method

If Pay Later

Admin enters a Promised Payment Date

End If

End If

Admin attaches pickup photos

Click "Confirm Pickup"

If weight is valid, photos are attached, and the amount does not exceed the amount payable

System records the weight, final price, discount, payment, and photos; marks the booking as Picked Up

Else

Display validation error

End If

End If

If booking has Arrived at Hub

Admin marks the booking Out for Delivery

If the booking is not weighed, or a balance remains with no promised date and the sender is the payer

Display reason the booking cannot be dispatched

End If

End If

If next step is delivery

Click "Complete Delivery"

If a balance remains

Admin collects the remaining balance and payment method, or records a Promised Payment Date

End If

Admin attaches 1 to 3 delivery photos

Click "Complete Delivery"

If photos are attached and the balance is paid or a promised date exists

System marks the booking as Delivered

Else

Display validation error

End If

End If

If a cancellation request is pending

Admin reviews the customer's stated reason

Click "Approve & Cancel" or "Decline"

System updates the booking accordingly and notifies the customer

End If

If booking needs a later payment recorded

Click "Record Payment"

Admin enters amount, method, and optional reference/receipt

If the amount does not exceed the remaining balance

System records the payment

Else

Display error

End If

End If

If booking needs to move to a different trip

Admin selects a new Trip and enters a reason

Click "Reassign"

End If

```

End If
End

```

Notes: the amount check is "not more than the amount still payable". An overpayment is rejected, not kept as a tip. The pickup-photo requirement is enforced by the form; the 1–3 delivery-photo requirement is enforced by the database.

```

16. Walk-in Booking Process
Begin
  Display Booking Form
  Admin selects Route
  Admin enters Sender Details and Receiver Details
  Admin enters Package Details and states a Payment Preference
  Click "Create Booking"
  If input is valid
    System saves the booking, marks it as a walk-in booking, and generates a Tracking Number
    Display Success Message with Tracking Number
  Else
    Display validation error
  End If
  If the walk-in customer later registers
    Admin may link the booking to the registered customer account
  End If
End

```

```

17. Trip Management Process
Begin
  Display list of trips with Route, Date, and Capacity
  Click "Create Trip"
  Admin selects Route and enters Departure and Estimated Arrival dates
  System applies the default Capacity and Amount per Kilo from Company Information
  Admin optionally checks "Also email subscribed customers"
  Click "Create Trip"
  If input is valid, defaults are set, and no other trip on the same route departs on the same day
    System saves the new trip
    System assigns matching Pending bookings on the same route that fit the capacity
    Display success message
  Else
    Display validation error
  End If
  If Admin opens a trip
    Display assigned bookings and capacity usage
    If trip status is Scheduled, today is the scheduled departure day, and every assigned booking has been picked
up
      Admin optionally enters the Estimated Arrival at the destination hub
      Click "Start Trip"
      System records the actual departure time and marks the bookings In Transit
    Else
      Display reason the trip cannot start yet
    End If
    If trip status is In Progress
      Click "Mark Arrived"
      System records the actual arrival time at the hub and marks the bookings Arrived at Hub
    End If
    If trip status is Arrived and every assigned booking is Delivered or Cancelled and fully paid
      Click "Complete"
    Else
      Display reason the trip cannot be completed yet
    End If
    If Admin clicks "Reschedule"
      Admin enters new Departure/Arrival dates and a reason
      System notifies affected customers
    End If
    If Admin clicks "Cancel"
      Display confirmation prompt
    End If
  End If
End

```

Start-date rule (LIVE): a trip can start only on its scheduled departure day in Philippine time. If the date is still in the future, the admin must reschedule to start early. If the date has passed, "Overdue — Reschedule Required." A trip also cannot start while a booking awaits a cancellation decision.

```

18. Customer Management Process
Begin
  Display list of registered customers

```

```

Admin searches by name, email, phone, or province
If Admin selects a customer
    Display customer profile and booking history
End If
End

```

19. Sales and Reports Process

```

Begin
    Display Sales & Reports Page with a month selector
    Admin chooses a month
    Click "Generate Report"
    System selects the trips scheduled to depart in that month
    For each trip, System summarizes its currently assigned bookings: shipping fees, payments received, successful
    refunds, amounts still to collect, and cancelled bookings awaiting a settlement decision
    Display the monthly grand total and the per-trip details
    If Admin clicks "Print Report"
        System opens the print view of the report
    End If
End

```

Corrections: the current page has **one** monthly per-trip report, not three sections. It has **"Print Report" only**, no "Export PDF" button (the browser's print dialog may offer "Save as PDF"). It reports payments and balances, **not profit**.

20. Announcements Process

```

Begin
    Display list of published announcements
    Click "New"
    Admin enters Title and Content, and selects or confirms a Category
    Admin optionally checks "Send Email"
    Click "Publish"
    If Title and Content are provided
        System publishes the announcement and notifies customers
        If "Send Email" was checked
            System queues the email to subscribed customers
        End If
        Display success message
    Else
        Display "Please enter a title – this is what customers see first" or "Please write the announcement content."
    End If
    If Admin clicks "Delete" on an announcement
        Display confirmation prompt
        If confirmed
            System removes the announcement
        End If
    End If
End

```

21. Inbox Process

```

Begin
    Display list of customer conversations with status (Bot Active, Waiting, Waiting on Customer, Resolved)
    Admin selects a conversation
    Display message thread
    Admin enters a reply
    Click "Send"
    System delivers the reply and marks the conversation as Waiting on Customer
    If the customer replies again
        Conversation returns to Waiting
    End If
    If Admin marks the conversation Resolved
        System marks it Resolved
    End If
    If a Waiting on Customer conversation receives no reply for 7 days
        System automatically marks it Resolved
    End If
End

```

22. Contact Inquiries Process

```

Begin
    Display list of public contact-form submissions with status tabs (New, Read, Resolved)
    Admin selects an inquiry
    Display inquiry details
    Admin optionally assigns the inquiry

```

```

Click "Resolved" (or update status)
System updates the inquiry status
End

```

```

23. Customer Feedback
Begin
  Display list of customer reviews with rating and message
  Admin filters by rating or searches
  If Admin clicks "Hide" on a review
    System hides the review from public display
  Else If Admin clicks "Restore"
    System makes the review visible again
  End If
End

```

```

24. Activity Logs Process
Begin
  Display log of Date & Time, Admin, Module, Action, and Details for the last 7 days
  Admin filters by search text, by Module, or hides sign-in entries
  If Admin clicks "Clear Filters"
    System resets all filters
  End If
  If Admin clicks "Export CSV"
    System generates a CSV of the filtered log
  End If
End

```

```

25. Company Information Process
Begin
  Display Company Information Page with tabs: Basic Info, Contact Info, Why Choose Us, Coverage Areas, Pricing
  Admin selects a tab and edits its fields
  Click "Save Changes"
  If input is valid
    System saves the updated company information
    Display "Changes saved successfully" message
  Else
    Display validation error
  End If
End

```

Note: the default rate and capacity in the Pricing tab are applied to trips created afterward; existing trips keep their own rate.

```

26. Storage Monitoring Process
Begin
  Display Photo Storage and Email Service health status
  Display booking photo folders
  If Admin selects eligible photos and confirms deletion
    System permanently deletes the selected photos and records the result
  End If
End

```

```

27. Admin Profile Process
Begin
  Display admin account information
  If Admin selects "Change Email"
    Admin enters New Email Address and Current Password
    Click "Update Email"
    Display "Check your new inbox" message or error
  Else If Admin selects "Change Password"
    Admin enters Current Password, New Password, and Confirm Password
    Click "Update Password"
    Display success or error message
  End If
End

```

Correction: the manuscript's Process 27 has an empty **If ... End If**.

```

28. Package QR Labels Process (proposed new)
Begin
  Admin opens a booking
  Admin sets the number of boxes (1 to 50)
  Click print labels
  System prints one QR label per box showing the Tracking Number and "Box n of total"
  If a label is scanned with a phone camera
    If the Admin is signed in
      Open the booking and display the box number
    Else
      Redirect to Login Page
    End If
  End If
End

```

Scope: all boxes belong to **one** booking and share one status. Scanning only opens the booking; it does not update status or confirm delivery. The system has no built-in scanner.

```

29. Cancellation Settlement Process (proposed new)
Begin
  Admin opens a Cancelled booking that has received payments
  Display money received, successful refunds, refunds in progress, and the current settlement decision
  If no decision has been recorded
    Display the money as awaiting review
  End If
  Admin selects "Full refund" or enters an amount to retain
  If an amount is retained
    Admin confirms that the customer agreed to it
  End If
  Click save
  If the retained amount does not exceed the money received and the customer agreement is confirmed when required
    System saves the decision and its history
    If money is still due to the customer
      Admin issues a refund through PayMongo or records a manual cash/GCash refund
    End If
    Display the settlement status
  Else
    Display validation error
  End If
End

```

```

30. Automated System Tasks (proposed new)
Begin
  Every day at 8:00 AM
    System emails payment reminders for bookings with a balance whose promised payment date has arrived
  Every minute
    System sends pending push notifications to registered devices
  Every 5 minutes
    System checks PayMongo for refunds that need to be recorded
  Every day
    System resolves support conversations waiting 7 days for a customer reply
    System permanently deletes activity logs older than 7 days
    System permanently deletes pickup and delivery photos of bookings delivered or cancelled more than 6 months
    ago, except featured deliveries
End

```

9. Verified computations and worked examples

All formulas below come from live function definitions (`order_payable_amount`, `update_order_payment_totals`, `derive_payment_status`, `effective_trip_price`, `record_pickup_payment`, `record_delivery_payment`, `get_cancellation_settlement_summary`), and are confirmed by the isolated tests in Section 12.3.

9.1 Pricing and balances

Quantity	Rule
Rate	The trip's <code>price_per_kg</code> ; if the trip's rate is 0, the current default rate from Company Information is used.
Shipping cost (gross)	<code>actual_weight × rate</code> , set when the actual weight is recorded at pickup. Before pickup it is 0, meaning not yet priced , not "paid".

Quantity	Rule
Discount	Set only by an admin before pickup is confirmed, and only if no payment has been recorded. Between 0 and the shipping cost. Requires a reason (Regular customer, Negotiated price, Other + notes).
Amount payable	$\max(0, \text{shipping_cost} - \text{discount_amount})$.
Money received	Sum of confirmed <code>payment_transactions.amount</code> .
Money returned	Sum of <code>payment_refunds.amount</code> with status succeeded only.
<code>amount_paid</code>	$\max(0, \text{money received} - \text{money returned})$.
<code>remaining_balance</code>	$\max(0, \text{amount payable} - \text{amount_paid})$.
<code>payment_status</code>	unpaid if nothing is held; paid if held $\geq$ payable (half-centavo tolerance); otherwise partial .

There is **no additional-charge field** and **no tip feature**. `package_quantity` does not change the price.

Example 1 — partial payment with refund. 12.5 kg $\times$ ₱80 = ₱1,000. Discount ₱100 (Regular customer) $\rightarrow$ payable ₱900. Payments ₱300 at pickup + ₱200 later = ₱500 received. A ₱100 refund succeeds $\rightarrow$ `amount_paid` ₱400, `remaining_balance` ₱500, `payment_status` partial. A GCash checkout of ₱200 that is still pending adds ₱0 until PayMongo confirms it.

Example 2 — full discount. 2 kg $\times$ ₱50 = ₱100; discount ₱100 $\rightarrow$ payable ₱0. No payment row is needed. The booking counts as priced because weight is recorded.

Example 3 — overpayment blocked. Payable ₱700, already received ₱200. Entering ₱501 is rejected: "The entered amount of ₱501.00 exceeds the amount still payable by ₱500.00... Extra money is not automatically recorded as a tip."

Example 4 — boundary cases. ₱0 or blank at pickup = nothing collected now. A negative amount is rejected. Repeating the same request (same idempotency key) creates no second payment. Reusing a GCash reference already recorded on another booking is rejected.

Payment attempt vs confirmed payment. A GCash checkout creates a `payment_attempts` row. Only when PayMongo's webhook or the verified return confirms it does the system create a `payment_transactions` row. Returning from the checkout page alone proves nothing.

9.2 Dispatch and delivery gates

- **Out for Delivery** is blocked if the booking has no weight, or if the sender pays, a balance remains, and no promised payment date is recorded. A receiver-pays (collect-on-delivery) booking is exempt.
- **Delivered** needs 1–3 photos, and either the balance is fully paid or a promised payment date exists.
- **Trip Complete** needs every booking Delivered or Cancelled **and** no non-cancelled booking with a remaining balance. So a booking delivered on a promise must be paid before its trip can be completed.

9.3 Cancelled bookings

Cancelled is a shipment status. The booking's old shipping fee is **not** collectible after cancellation. The settlement summary separates:

- money received (confirmed payments);
- money already refunded (succeeded);
- refunds still pending (creating/pending/processing);
- the customer-agreed retained amount (only when a `retained_fee` decision with confirmed agreement exists);
- refund still due = received – refunded – agreed retained amount;
- **For Review** when no decision exists yet.

Example 5. A cancelled booking received ₱600. ₱200 was refunded successfully, ₱150 is pending, and the admin recorded ₱100 retained with the customer's agreement. Money still held = ₱400. Refund still due = $600 - 200 - 100 = ₱300$, of which ₱150 is in progress and ₱150 not yet started. No shipping balance can be collected. If no decision exists, the ₱400 is shown as awaiting review and **must not be called a cancellation fee**.

9.4 Trips

Term	Field	Meaning
Scheduled departure	<code>departure_date</code>	Planned departure; its Philippine calendar day is the only day the trip can start.
Planned arrival	<code>arrival_date</code>	Must be after departure. Not the actual arrival.
Actual departure	<code>departure_at</code>	Stamped by the server when "Start Trip" succeeds.
Estimated hub arrival	<code>estimated_arrival_at</code>	Optional, entered at start, must be after actual departure.
Actual hub arrival	<code>arrived_at</code>	Stamped by the server at "Mark Arrived". Bookings become Arrived at Hub .
Delivery to receiver	per booking	Later, one booking at a time: Out for Delivery $\rightarrow$ Delivered. Hub arrival is not delivery .

Capacity. Trip load = sum of actual weights of its non-cancelled bookings. For a trip with capacity above 0, bookings are refused when load would exceed **capacity + 200 kg** allowance. At booking time the weight is unknown (0), so choosing a trip does not reserve kilograms; the weight check applies again at pickup. The browser's auto-assignment on trip creation stops at the planned capacity (without the allowance). The server's multi-user race safety was not tested.

9.5 QR labels

Each label encodes a link `/admin/orders/{booking_id}?box=n` and shows the booking's tracking number. It identifies the **booking** and box number only. No per-box table, scan history, receiver identity check, or automatic status change exists. The QR shown during GCash checkout is a PayMongo payment link, not a cargo label.

9.6 Reports

- **Current report page** (`/admin/sales` and `/admin/reports` → `PerTripSalesPage`): the admin chooses a month; the report includes trips whose `departure_date` is in that month and the bookings **currently** assigned to them. Payments and refunds of those bookings count **regardless of their payment date**. A reassigned booking appears only under its current trip. A cancelled booking with no trip cannot be placed in a trip report. Totals: shipping fees, payments received, money returned, payments after refunds, amount still to collect, pending/failed/uncertain refunds, cancelled money awaiting a decision.
- **Date-range report function** `get_financial_report_data(start, end)` exists in the database and in `database.js`, grouping payments by payment date and refunds by `succeeded_at`. The page that used it (`ReportsPage.jsx`) is **not mounted** at any route. Do not describe it as a current screen.
- Neither report records expenses or computes **profit**. Use "payments received" or "collections", never "profit" or "net income".

9.7 Customer support

The assistant ("CargoMate PH") is **menu-driven with fixed keyword/pattern rules** in `src/lib/supportChatEngine.js`, running in the customer's browser. Main menu: My bookings, Payment and refunds, How to book, Shipping information, Talk to an admin. Sub-menus include Ways to pay; Payment details for a booking; Cancellation and refund guidance; Shipping rates; Service areas; Packaging; Restricted items; Pickup and delivery process; Contact information. Typed messages are matched against prepared English, Tagalog, and Bisaya phrases. Booking answers use only the signed-in customer's own records. Urgent keywords or "Talk to an admin" hand the conversation to the admin inbox. **No AI model or external language service is called**. Describe it as a "menu-driven automated assistant", not a chatbot that understands natural language.

10. Automation, retention, and deletion inventory (LIVE)

All 10 cron jobs below are **active** and succeeded in the last 2 days (weekly job last ran Sunday 20 Sep). Times are UTC; Philippine time is UTC+8. "Succeeded" means the scheduled SQL ran and, where it calls an Edge Function, the call was sent. All recent Edge Function calls returned HTTP 200. Email and push acceptance by outside providers was not independently confirmed.

Job (schedule, UTC)	What it does	Records/files affected	Eligibility	Permanent?	History kept
<code>daily_payment_reminders</code> (0 0 * * * = 8:00 AM PH)	Calls <code>process-daily-reminders</code> to email payment reminders	Emails; stamps <code>orders.last_reminder_sent_at</code>	<code>remaining_balance > 0</code> , promised date today or earlier, not reminded today. No status filter (Section 12, issue 3)	No deletion	Order unchanged except stamp
<code>process_push_deliveries</code> (every minute)	Sends queued push notifications	<code>notification_delivery_jobs</code> status	Due jobs; skipped if no device	No	In-app notification stays
<code>monitor_push_delivery_health</code> (every 5 min)	Checks push queue health	Admin alert if stuck	—	No	—
<code>paymongo_refund_recovery</code> (every 5 min)	Re-checks PayMongo for refunds	<code>payment_refunds</code> , recovery jobs	Eligible jobs	Queue rows temporary	Refunds and payments kept
<code>photo_storage_health_check</code> (15 */6 * * *)	Storage health check	<code>photo_storage_events</code>	—	No	—
<code>scheduled_old_photo_cleanup</code> (30 1 * * * = 9:30 AM PH)	Deletes old evidence photos (Edge Function named "archive..." but it deletes)	Pickup/delivery photo files and their references	Booking Delivered or Cancelled more than 6 months ago (latest matching status event); not featured; receipts protected	Yes, permanent	Booking, payments, timeline kept
<code>purge_old_activity_logs</code> (0 3 * * *)	Deletes old audit rows	<code>activity_logs</code>	Older than 7 days	Yes	—
<code>purge_old_notification_delivery_jobs</code> (20 3 * * *)	Deletes finished	<code>notification_delivery_jobs</code>	Completed, older than 30 days	Yes	In-app notifications

Job (schedule, UTC)	What it does	Records/files affected	Eligibility	Permanent?	History kept
	push jobs				kept
<code>purge_photo_storage_operational_logs</code> (0 3 * * 0, Sundays)	Deletes storage logs and finished cleanup tasks	<code>photo_storage_events</code> (>30 days), completed <code>photo_cleanup_queue</code> (>7 days)	As stated	Yes	Pending tasks kept
<code>auto_resolve_stale_conversations</code> (30 3 * * *)	Resolves idle threads	<code>conversations.status</code>	<code>waiting_customer</code> with no customer message for 7 days	Status change only	Messages kept

Other automatic work (event-driven, not scheduled): notification triggers on bookings, trips, payments, refunds, feedback, inquiries, and messages; the push worker is woken when new jobs are queued; announcement email broadcast (durable queue with retries); trip reschedule email; PayMongo webhook reconciliation. Admin-selected photo deletion (Storage Monitoring) is manual and permanent.

Not automatically deleted: bookings, trips, payments, refunds, status events, settlements and their history, notifications, chat messages, feedback, inquiries. The 60-day announcement filter in `database.js` only limits what is displayed; it deletes nothing. `supabase/migrations_archive/` is not active.

11. Required diagram changes

These are instructions for a future manuscript edit. No diagram has been edited.

Figure (PDF page)	Change
Fig. 11–20 (pp. 6–8)	Keep as the present manual system . Do not add database stores.
Fig. 21 (p. 14)	Keep actors Customer, Public User, Administrator. Customer → system: add "Contact Detail Corrections" and "GCash Payment". Admin → system: add "Discount", "Trip Start/Arrival", "Cancellation Settlement/Refund", "Package Labels". System → Customer: add "Refund Status". Optionally show PayMongo as an external entity (GCash payments/refunds). Do not show receiver verification.
Fig. 22 (p. 15)	Registration writes <code>profiles_db</code> and <code>legal_consent_db</code> .
Fig. 23 (p. 15)	Add a read arrow <code>trips_db</code> → Cargo Booking (trip open/capacity check). Booking writes <code>orders_db</code> only. Change the arrow to <code>trips_db</code> from a write to a read. Add note "No weight or price at booking; out-of-coverage flagged For Review."
Fig. 24 (p. 16)	Keep reads from <code>orders_db</code> and <code>order_status_events_db</code> ; add read from <code>trips_db</code> (trip timing). Correct the store label to <code>order_status_events_db</code> . Output "Tracking Timeline (names masked)".
Fig. 25 (p. 16)	Input "Trip Details / Reschedule Reason / Cargo Assignment". Read <code>company_information_db</code> (default rate and capacity).
Fig. 26 (p. 16)	Inputs "Actual Weight, Discount, Payer, Payment, Pickup Photos". Also write <code>payment_transactions_db</code> (pickup payment).
Fig. 27 (p. 16)	Add <code>payment_attempts_db</code> (GCash checkout attempt) before <code>payment_transactions_db</code> ; add <code>payment_refunds_db</code> . Label: "only confirmed payments are recorded".
Fig. 28 (p. 16)	Correct <code>orders_status_events_db</code> → <code>order_status_events_db</code> . Add <code>trips_db</code> write (actual departure/arrival). Label outputs "In Transit / Arrived at Hub".
Fig. 29 (p. 16)	Inputs "Delivery Photos (1–3), Remaining Payment or Promise Date".
Fig. 30 (p. 17)	Wrong input label: replace "Cargo Delivery and Completion" with "Cancellation Request and Reason". Show Administrator as the decision source with flow "Approve/Decline Decision". Add <code>cancellation_settlements_db</code> and <code>payment_refunds_db</code> .
Fig. 31 (p. 17)	Add "only after Delivered; one review per booking".
Fig. 33 (p. 17)	Correct <code>payments_transactions_db</code> → <code>payment_transactions_db</code> . Add reads of <code>trips_db</code> , <code>payment_refunds_db</code> , <code>cancellation_settlements_db</code> . Input "Month". Output "Monthly Per-Trip Report (printable)".
Fig. 34 (p. 17)	Input "Company, Coverage, Default Rate and Capacity".
Fig. 35 (p. 17)	Keep. Optionally add <code>email_subscriptions_db</code> for the email option.
Fig. 36 (p. 18)	Customer input "Menu Choice / Support Message". Label process "Menu-Driven Assistant and Admin Handoff". Reads <code>orders_db</code> (customer's own bookings).

Figure (PDF page)	Change
Fig. 37 (p. 18)	Keep. Visitor output: "Confirmation Message".
Fig. 38 (p. 48)	Public tree: add "FAQ", "Unsubscribe".
Fig. 39 (p. 49)	"Book Shipment", "Enter Sender/Receiver Details", and "Enter Package Details" are wrongly placed under Track Package ; move them under a separate "Book Shipment" branch. Add "Payment History" and "Order Detail (Pay, Cancel, Review)". Fix spelling "Send/Receice" → "Send/Receive".
Fig. 40 (p. 50)	Bookings branch lists "Display Booking List" twice; replace with Display Booking List, Create Booking, Display Booking Details, Print Package Labels. Sales & Report: replace the three children with "Monthly Per-Trip Report". Fix "Send/Recieve" → "Send/Receive".

Optional new ERD. If the adviser requires an entity-relationship diagram, draw the 13 core tables with the Section 5.14 cardinalities and caption it: **"Figure X. Entity Relationship Diagram of the Core Business Tables of CargoExpress PH (13 of 29 application tables; supporting and technical tables are omitted)."** The count 29 is verified live.

12. Unresolved questions and deployment uncertainties

12.1 Issues requiring an owner decision before final manuscript wording

These are **implementation issues**, not intended business rules. Do not document them as policy.

- Bot messages can be forged (LIVE).** The deployed `guard_chat_message_insert` lets any signed-in customer insert a message labelled `bot`. Assistant replies are produced in the browser and saved under the customer's identity. A technically capable customer could post a fake "assistant" message into their own thread; the admin would see it labelled as the bot. It cannot affect other customers' threads, bookings, or payments. The 24 Sep secure version was reverted, and an orphan `support-bot` Edge Function remains deployed. **Manuscript wording until resolved:** "the assistant's replies are recorded in the conversation", without claiming server-side bot authorship.
- Out-of-coverage bookings are flagged, not held (SOURCE, high confidence).** The booking form sends status `Pending Review`, but the server's insert trigger overwrites it with `Pending` (or `Assigned` if a trip was chosen). Only `service_area_status = 'for_review'` marks it. Nothing on the server blocks assignment, pickup, or trip start while a booking is For Review, and new-trip auto-assignment picks flagged bookings too. The manuscript (p. 10) says such bookings are "placed under review and evaluated ... before they continue". **Decide:** either change the manuscript to "flagged for administrator review" (documentation only), or fix the system to hold them. Live has no such booking now, so this was not observed live.
- Payment reminders for cancelled bookings (SOURCE; latent live).** `process-daily-reminders` does not exclude `Cancelled`, and `update_order_payment_totals` does not zero a cancelled booking's `remaining_balance`. A cancelled booking with a balance and a promised date would get reminder emails. Live currently has 0 cancelled bookings, so it has not occurred. Do not document reminders for cancelled bookings.
- Report month grouping near midnight (SOURCE; needs verification).** `PerTripSalesPage` groups trips by the first 7 characters of `departure_date` as returned by the API. If the API returns UTC (usual for Supabase), a trip departing before 8:00 AM Philippine time on the 1st of a month would be counted in the previous month. Verify with a staging trip before describing the month rule in detail.
- Pickup photos required only by the form.** Delivery photos (1–3) are required by the database; pickup photos only by the browser form. Documentation-safe wording: "the pickup form requires at least one photo".

12.2 Remaining unverified facts

- Deployed Vercel frontend version vs local source.
- Whether each deployed Edge Function's code equals local source (versions exist; content not compared).
- Email provider delivery and push delivery to real devices (queue results are live; provider acceptance is not).
- PayMongo webhook configuration on the PayMongo side.
- Multi-user race behavior (tests are single-connection).
- Adviser acceptance of the 13-table scope and whether an ERD is required.

12.3 Tests run for this handoff (all passed; none touched production)

Test	Type	Result
<code>scripts/support-chat-engine-contract-test.mjs</code>	Source contract	Passed
<code>scripts/per-trip-sales-report-test.mjs</code>	Source/fixture contract	Passed
<code>scripts/pickup-payment-amount-pgtest/run.mjs</code>	Isolated PGLite database	60/60 passed
<code>scripts/trip-start-dates-pgtest/run.mjs</code>	Isolated PGLite database	26/26 passed
<code>scripts/cancellation-settlement-pgtest/run.mjs</code>	Isolated PGLite database	23/23 passed (lock-order checks are structural only)

No browser test and no full migration replay were run. The working tree was checked afterward and was unchanged.

13. Prioritized manuscript update checklist

- Database Design introduction (p. 51).** Replace with: "The following tables present the core business database design of CargoExpress PH. They describe the records used in booking, trip, payment, refund, cancellation, communication, and feedback processes. The system also uses supporting

tables for configuration, audit logs, legal consent, and delivery queues, and platform-managed tables for authentication and file storage; these are summarized separately. Therefore, the tables below do not represent the complete physical database, which contains 29 application tables."

2. **Replace Tables 1–14** with the 13 tables of Section 5 in business-flow order, adding **Cancellation_settlements**. Keep the three-column grid. Move Activity_logs (and optionally Company_information) to a supporting paragraph listing the Section 4.2 tables.
3. **System Rules (pp. 9–13):**
 - p. 10: "placed under review ... before they continue" → "flagged for administrator review" (or wait for the fix; Section 12, issue 2).
 - p. 10: trip creation — capacity and rate come from Company Information defaults; one trip per route per day.
 - p. 11: add discount at pickup; payment only up to the amount payable; confirmed payments only; successful refunds reduce the amount paid.
 - p. 11: trips start only on the scheduled Philippine date; overdue trips must be rescheduled; "Arrived at Hub" is not delivery.
 - p. 12: dispatch requires weight and payment/promise (receiver-pays exempt); delivery needs 1–3 photos.
 - p. 12: support is menu-driven with prepared answers in English, Tagalog, or Bisaya.
 - Add a paragraph on cancellation settlement and refunds.
4. **Process specifications:** replace 6, 8, 9, 11, 15, 17, 19, 20, 21, 22, 24, 27 with Section 8; minor updates to 1, 2, 3, 5, 12, 13, 25, 26; optionally add 28–30.
5. **Diagrams:** apply Section 11. Priority fixes: Fig. 30 wrong input label; Fig. 33 and Fig. 28 store-name typos; Fig. 39 misplaced booking branch; Fig. 40 duplicate node.
6. **Test Cases (pp. 63–72):** Test Case 5 output should mention both "Enter a tracking number to search" and "Shipment Not Found". Test Case 6's screenshot shows editable Capacity and Amount per Kilo, which are now read-only defaults; retake it. Test Case 9's rule is correct (delivered or cancelled and no unpaid balance).
7. **Interface Design (pp. 80–93):** Preview 18 is missing (numbering jumps 17 → 19). Preview 24 caption "Sales and Reports Page" should be "Announcement Management". Preview 17's description ("retrieves real-time database information ... instantly") → "menu-driven assistant that shows the customer's own booking information".
8. **Revision of the System (p. 73):** "Separate the full name fields into First Name and Last Name" applies to **booking** sender/receiver, not account registration (still one full name). "QR code system for tracking parcels" → "QR labels that link each box to its booking". "Filter and data per month" matches the current monthly per-trip report.
9. **Cost-Benefit Table 15 (p. 78):** the listed amounts add up to ₱1,888.00 (₱100 + ₱1,788), not ₱2,480.32. Correct the total or add the missing item.
10. **Software Requirements (p. 76):** push uses Firebase Cloud Messaging and standard Web Push (VAPID). Server-side logic also runs in Supabase Edge Functions (Deno). "Generating PDF reports" (Peopleware) → "printing reports".
11. **Chapter 3 Recommendation 5 (p. 96):** "route analysis" is correctly future work; keep it there.
12. **Resolve Section 12.1 issues 1–3** with the owner before finalizing support and coverage wording.

Are database changes necessary? No schema change is needed to revise Chapter 2. The work is **documentation only**. Section 12.1 lists three behavior issues (bot-message forgery, coverage review not holding bookings, reminders on cancelled bookings). Fixing them is optional, separate from the manuscript, and should be reviewed and tested on its own.

14. Simple Taglish explanation for the project owner

Ano ang ginawa: Tiningnan ko ang Chapter 2, ang code, at ang **live Supabase database mismo** (read-only lang, walang binago). Lahat ng 219 migrations ay naka-apply na sa live database, kaya ang nakasulat dito ay ang totoong gumagana ngayon.

Database: May **29 tables** ang system. Para sa Chapter 2, **13 core tables** ang ilagay: Profiles, Trips, Orders, Order_status_events, Payment_transactions, Payment_refunds, **Cancellation_settlements** (bago, wala pa sa manuscript), Announcements, Notifications, Conversations, Chat_messages, Contact_inquiries, at Customer_feedback. Ang iba (activity logs, company info, email queues, push, photo cleanup, atbp.) ay i-summarize lang sa isang paragraph. Kailangan pa rin sila ng system, huwag burahin. Maraming kulang na columns sa manuscript, lalo na sa Orders (63 columns ngayon, 44 lang ang nakalista), kaya gamitin ang Section 5.

Bayad: Timbang × rate = shipping fee. Bawas ang discount = babayaran. Ang "paid" ay confirmed payments lang, bawas ang successful refunds. Hindi puwedeng sobra ang bayad (walang tip). Kapag cancelled, **hindi na singilin** ang lumang balance. Kailangan ng admin decision: full refund, o may maiiwanag halaga kung pumayag ang customer.

Trips: Sa mismong scheduled na araw lang puwedeng i-start. Kapag lumampas, kailangan i-reschedule. Ang "Arrived at Hub" ay dating sa bodega, hindi pa delivery sa receiver.

Support chat: Menu at nakahandang sagot (English, Tagalog, Bisaya) na may "Talk to an admin". **Hindi ito AI**. Kapag resolved na at nag-message ulit ang customer, bagong bot session ang magsisimula, hindi bumabalik sa parehong admin.

Report: Isang monthly per-trip report, may "Print Report" pero walang "Export PDF" button, at walang profit computation.

Mga dapat ayusin o pagpasyahan (hindi ito policy, mga isyu ito):

1. Puwedeng mag-send ang customer ng pekeng "bot" message sa sarili niyang chat, dahil na-rollback ang security fix noong Sept 24.
2. Ang out-of-coverage booking ay naka-flag lang na "For Review", hindi talaga naka-hold. Puwede pa rin itong ma-assign at ma-pickup.
3. Puwedeng makatanggap ng payment reminder ang cancelled booking na may natitirang balance. Wala pang ganitong kaso sa live ngayon.

Para sa thesis, **documentation lang ang kailangang baguhin**. Hindi kailangang magbago ng database para lang tumugma sa manuscript.